

Rocket GNC Monitor v0

Complete functionality and developer documentation
Application version 0.1.0

MIT Rocket Team
Avionics and GNC

21 September 2026

Document control and reading guide

This report documents the implemented desktop application in `ui_work/rocket_UI_v0`, including Digital/Analog USB video, the portable flight-file workflow and URRG launch-site preset. Part I explains the operator interface and its observable behavior. Part II describes the software architecture, contracts, source files, build process and extension points. Appendices provide configuration, sample and file-field references.

Item	Baseline
Documentation date	21 September 2026.
Software identity	Rocket GNC Monitor, Python package version 0.1.0, application generation v0.
Implementation reviewed	Source files and checked-in resources under <code>rocket_UI_v0</code> ; exact file hashes are recorded in <code>evidence/source-manifest.json</code> .
Current distribution	macOS arm64 app and DMG, rebuilt for Digital/Analog video, flight save/load and URRG. Windows makefiles remain available; the Windows ZIP predates these changes.
Validation evidence	77-test Mac regression run for dual video, focused recording/replay checks and packaged startup/demo/flight/OpenRocket verification. Historical September 20 results are retained separately. Physical dual-camera acceptance remains outstanding.
Report style	KOMA-Script, 10 pt Latin Modern, A4 with 18 mm margins, blue headings, shaded tables and code blocks, matching the supplied GenMAPS report. All diagrams are editable TikZ with LaTeX text.

Reading the evidence Implemented behavior is distinguished from nominal simulation assumptions and future controlled-vehicle work. A valid software test result is not a measured mechanical pose, a radio acknowledgment or field acceptance.

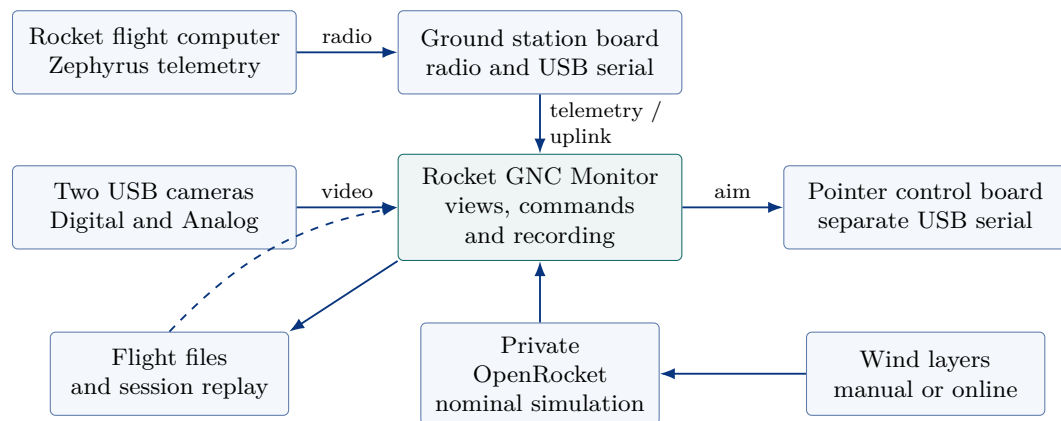


Figure 1: System boundary. The two serial boards are independent. Simulation, replay and drawing operate inside the desktop application; the app does not infer unavailable actuator feedback. Dashed flow denotes offline restoration.

Contents

Document control and reading guide	2
I. Application functionality and operation	5
1. Purpose, scope and operating model	5
1.1. Supported machines and offline use	5
1.2. What the app establishes	5
2. A complete operating workflow	6
2.1. Before connecting equipment	6
2.2. Receiving and recording	6
2.3. After the flight	6
3. Navigation and connection control	6
3.1. Rocket radio status	7
3.2. Serial behavior and command availability	7
4. Flight overview, coordinates and time	7
4.1. Metrics and plots	7
4.2. Altitude conventions	8
4.3. Clock meanings	8
5. Digital and Analog video	8
6. Antenna assembly and pointing	9
6.1. Viewing the physical model	9
6.2. Manual controls and the meaning of ZERO	9
6.3. Ground-GPS freeze and automatic tracking	9
7. Rocket commands, power and recovery	10
7.1. Telemetry subview	10
7.2. Rocket commands subview	10
7.3. Power subview	10
7.4. Recovery and pyro subview	10
8. Mission configuration and launch sites	11
8.1. Latitude/longitude, MGRS and Plus Codes	11
8.2. URRG preset	11
8.3. Mission JSON versus portable flight files	11
9. Wind and nominal OpenRocket simulation	11
9.1. Manual and imported wind	11
9.2. Online weather	12
9.3. Running a reference	12
9.4. Importing a reference and interpreting results	12
10. Recorded Zephyrus demonstration	12
11. Logging, sessions and portable flight files	13
11.1. Starting and stopping logging	13
11.2. Session-folder replay	13
11.3. Save flight and Open flight	13
11.4. Storage and recovery	14
12. Alerts, diagnostics and troubleshooting	14
13. Keyboard shortcuts and operational limits	15
II. Codebase and developer documentation	16
14. Repository, entry points and source ownership	16
14.1. Module map	16
14.2. Dependencies and resource resolution	17
15. Runtime architecture and state ownership	17
15.1. Concurrency model	18
15.2. Connection generations and asynchronous results	18
15.3. Mode transitions	18
15.4. Lifecycle and shutdown	19
16. Data contracts, units and geographic frames	19
16.1. Mission and Sample	19

16.2. WGS84 and local ENU	19
16.3. Time normalization and reboot behavior	20
17. Serial and protocol implementation	20
17.1. Port ownership and transport rules	20
17.2. Telemetry framing and checksum	20
17.3. Pointer command packets	21
17.4. Rocket uplink packets	22
18. Recording, replay and portable archive internals	22
18.1. Session writer and durability	22
18.2. Raw capture format	23
18.3. Replay reader	23
18.4. Flight archive layout	23
18.5. Active snapshot consistency	24
18.6. Archive validation and application	24
18.7. Read-only analysis example	25
19. Legacy import and demo provenance	25
20. Video implementation and synchronization	25
21. Weather and OpenRocket bridge contracts	26
21.1. Weather normalization	26
21.2. Trajectory contract	26
21.3. Simulation job boundary	26
21.4. Reproducibility and interpretation	27
22. UI, font and antenna graphics maintenance	27
22.1. UI responsibilities	27
22.2. Where the antenna's physical description is defined	27
22.3. Changing the model without changing hardware behavior	28
23. Developer setup and native packaging	28
23.1. Source environment	28
23.2. Runtime and bridge	28
23.3. Native platform builds	29
23.4. Package contents and artifacts	29
24. Testing, verification and evidence	29
24.1. Test organization	29
24.2. Legacy parity baseline	30
24.3. Packaged verification	30
24.4. Recorded results and their limits	31
25. Extension recipes and maintenance boundaries	31
25.1. Add another launch-site preset	31
25.2. Support the new three-axis telemetry board	31
25.3. Add a command or pointer feedback protocol	31
25.4. Extend a saved schema	32
25.5. Change simulation physics	32
25.6. A practical change checklist	32
A. Complete mission configuration reference	33
B. Complete normalized sample reference	34
C. Legacy CSV columns and derived values	35
D. Worked file examples	36
D.1. URRG planning template	36
D.2. Wind profile	36
D.3. Normalized trajectory	37
D.4. Portable flight metadata excerpt	37
E. Source and validation index	37
F. Glossary	38

Part I.

Application functionality and operation

1. Purpose, scope and operating model

Rocket GNC Monitor is a native desktop ground application for an amateur rocket. It combines two independent serial links, Digital and Analog USB video, recorded-flight playback, a visual antenna assembly, telemetry and actuator displays, and a nominal OpenRocket trajectory. The current transport contract is Zephyrus. The interface has places for a configurable number of canards and four fin tabs, but the Zephyrus packet contains four legacy servo drives; it does not provide the new vehicle's requested and measured canard/tab angles.

The application starts in **LIVE**, on **Control panel**, without opening either serial port, starting polling or opening a camera. Connect each board explicitly. A ground-board USB connection, a working rocket radio link and a pointer-board USB connection are three separate observations. A screen showing telemetry does not imply that an uplink command was acknowledged by the rocket.

Mode	Source and controls	Persistence and transition behavior
LIVE	Physical Zephyrus telemetry; ground commands require its board connection. Pointer manual controls require the pointer board independently.	Logging captures received data. Source changes close physical transports and clear displayed history. Reconnect and restart polling explicitly.
DEMO	One complete bundled GS1, GS2 or GS3 receiver log; synthetic video test pattern. Commands are simulated.	No physical serial transport. Can record a rehearsal or export the complete selected dataset. Returning to LIVE restores the prior reference and retains mission settings.
REPLAY	Indexed session telemetry and recorded video, initially paused. Physical commands disabled.	Seek, step, change speed and export. Session files are read for replay; a new flight archive can preserve the session and displayed position.

The three source modes are not launch states. Rocket states such as Preflight, Flight and Main come from telemetry and are shown within a mode. Likewise, **Start Logging** is independent of **Start Polling**: opening a recording does not by itself create telemetry.

1.1. Supported machines and offline use

The current verified build is macOS arm64. The distributable app includes Python, Qt/PySide6, NumPy, serial support, FFmpeg, the private Java runtime, the team's OpenRocket engine and bridge, motors, demo recordings and Lucida Grande. A production machine does not need a separate Python or Java installation. The macOS app is packaged with a minimum-system declaration of macOS 13; that is not evidence that every such system has been tested.

Windows has native build entry points and a previously tested x64 package. The current Mac feature revision has not been rebuilt and qualified on Windows. Linux-related camera and runtime branches exist, but Linux is not a qualified release target. Keep the whole Windows package together; its executable alone is insufficient.

Serial reception, local video input, demonstration, recording/replay, coordinate conversion and simulation work offline when the package and inputs are present. Online wind retrieval requires internet access. USB devices may require an appropriate OS driver, and macOS camera permission must be granted.

1.2. What the app establishes

A command marked *sent* has been dispatched on the serial interface; neither legacy protocol provides the command-level acknowledgment needed to prove the requested physical action. The antenna graphic follows the last sent target, not measured motor encoders. Its dimensions are illustrative rather than surveyed CAD.

The OpenRocket output is a nominal reference. It does not qualify the new controlled rocket's aerodynamic, actuator or control-loop behavior. Integrated Zephyrus rotations are displayed as received and are not a validated attitude solution. Missing measurements are shown as unavailable rather than inferred.

2. A complete operating workflow

2.1. Before connecting equipment

1. Start the app and confirm **LIVE** on the connection panel. Both board cards should be disconnected.
2. Open **Mission** → **Configure**. Name the mission and enter the launch origin, or choose **URRG**. Enter the ellipsoid and mean-sea-level elevations separately, then select the verified legacy height interpretation.
3. In **Mission & wind**, enter and apply a manual wind profile or retrieve one for the intended UTC time. Select the rocket's `.ork` file and any additional motor curves. Run the nominal simulation if a reference is needed.
4. Save a planning `.rktflight` file so the settings, selected model/motors and available reference can be reopened together.

2.2. Receiving and recording

1. Refresh the two serial selectors. Assign the ground-station board and antenna-pointer board to different ports.
2. Connect the ground board and click **Start Polling**. Wait for **Rocket telemetry live**, then verify packet age and the displayed data.
3. Connect the pointer board if pointing control is needed. Manual pointer control is independent of the ground board.
4. On **Flight overview**, choose **Find cameras**, select the USB receiver and press **Start camera**. Confirm that frames are arriving.
5. Click **Start Logging**. It immediately creates a unique session folder. Check Diagnostics for the recording path and any loss/error indication.
6. For tracking, use **Send to AntPtr** to freeze the receiver GPS, then **Track live rocket**. Tracking requires both boards, polling and a usable fresh target. Review the meaning of ZERO and Stop tracking in Section 6.

2.3. After the flight

Stop tracking and stop logging. Use **Save flight** to create the portable archive; the save waits for pending recorder/video closure. Reopen the file to check the recorded session in paused replay. Retain the original session folder when preserving full capture provenance. Close the app normally to finalize its workers. A flight saved while logging is still active is a snapshot and can omit the unfinished video segment.

3. Navigation and connection control

The persistent header contains the mission name, source-mode selector, Start/Stop Polling and Start/Stop Logging. In LIVE and REPLAY the two serial cards remain visible; controls are enabled according to mode and board state. The status area provides local wall-clock time, source/status messages and alerts. The **View** menu selects the daylight theme; this changes contrast, not data.

Workspace	Function
Control panel	Ground USB, rocket radio and pointer USB status; packet age, RSSI, valid/rejected counts and readiness.
Flight overview	Main metrics, camera/file video, projected trajectory, vertical profile and attitude instrument.
Antenna pointer	Orbitable assembly, manual azimuth/elevation, direction commands, ZERO, ground-GPS freeze and tracking.
GNC & actuators	Three angular-rate and integrated-rotation traces, canard/tab channel slots and legacy servo drives.
Mission & wind	Mission configuration, launch location, wind layers, weather, rocket/motor selection and simulation.

Workspace	Function
Sessions & replay	Portable flight files, session-folder replay, legacy CSV import, sample export, seeking and event timeline.
Diagnostics	Decoded sample JSON, packet counters, recording state, events and alert acknowledgment.
Rocket controls	Four subviews: Telemetry, Rocket commands, Power, Recovery & pyros.

3.1. Rocket radio status

Status	Meaning	Normal next action
Disconnected	Ground USB transport is not connected.	Select and connect the telemetry board.
Paused	Ground USB is open but telemetry polling is stopped.	Start polling to receive; uplinks remain available.
Waiting	Polling is active but no fresh valid packet has established reception.	Check transmitter, receiver, antenna and serial source.
Receiving	A checksum-valid LIVE packet arrived within the configured freshness interval.	Monitor packet age and loss indicators.
Stale	Previously received valid telemetry has aged beyond that interval.	Restore reception; explicitly restart tracking if it stopped.

The default freshness interval is 2 s. Reconnecting the ground board clears prior reception evidence. A connected USB device alone never proves a rocket radio link. The trailer carrying ground-receiver GPS has no independent checksum.

3.2. Serial behavior and command availability

Both boards use 115200 baud, eight data bits, no parity, one stop bit and no flow-control override. DTR/RTS follow the original pyserial defaults. Ground reads use a 1 s timeout; pointer reads use 0.1 s. The selectors reject assigning the same serial device to both roles, including the usual macOS `/dev/tty.` versus `/dev/cu.` alias. A connected or connecting board's port is removed from the other selector and restored when disconnected.

Stopping polling leaves the port open and leaves rocket uplinks available. Disconnecting a board closes its transport and clears relevant pending state. There is no automatic reconnection, retransmission or tracking restart. A partially written packet is treated as uncertain, not retried.

4. Flight overview, coordinates and time

4.1. Metrics and plots

The six main metrics are altitude in metres, velocity in metres per second, aligned flight time in seconds, battery in volts, receiver RSSI in dBm and sample age in seconds. Missing values remain blank/unavailable. Battery is the sum of the three decoded cell voltages; velocity is the legacy reported integrated velocity rather than a recomputed derivative of the plotted altitude.

The trajectory view offers **Plan: East/North**, **Side: East/Up** and a fixed projected **Perspective**. The actual trace uses accepted local positions. The reference is dashed, with a diamond at the simulated position for the same aligned time. The residual label is the Euclidean three-dimensional distance between actual and reference positions at that time. It is not a closest-point distance or an estimate of navigation error.

A reference marker requires aligned time within the reference interval; the app does not extrapolate before or after the simulation. A trace or last point can remain visible when new packets are absent, so always read sample age. The vertical profile overlays reported altitude with the reference's launch-relative up coordinate. Their comparison depends on using the correct altitude convention.

4.2. Altitude conventions

For LIVE geographic-to-local conversion, configure an origin and a valid 3D rocket GPS fix. Choose one verified interpretation:

Mission choice	Conversion used for the rocket's absolute height
Unknown	Geographic trajectory position remains unavailable. Telemetry values still display.
GPS absolute ellipsoid	Use the reported GPS altitude directly.
GPS zeroed at launch origin	Add mission ellipsoid altitude to the reported GPS-relative altitude.
Filtered barometer relative to launch	Add mission ellipsoid altitude to the reported filtered barometric altitude.

Mean-sea-level elevation is a separate simulation/weather input; it is not substituted for ellipsoid height in the geographic transform. The legacy antenna tracking convention is separate again (Section 6).

4.3. Clock meanings

There are three primary clocks. Device time is flight-computer uptime in seconds after rollover extension. UTC is the packet's host reception time. Monotonic host time measures freshness and session elapsed time. Flight time is

$$t_{\text{flight}} = t_{\text{device}} - t_0.$$

In LIVE, observing a received Preflight-to-Flight transition establishes t_0 . Connecting mid-flight does not establish liftoff time. A detected device reboot clears time alignment and displayed history and stops tracking. The Sessions action **Set displayed sample to flight** $t = 0$ provides explicit manual alignment.

The attitude display uses the legacy integrated rotations. In GNC, axes are labeled X/roll, Y/pitch and Z/yaw. The attitude graphic and traces describe the available legacy data; they do not imply a quaternion navigation estimator.

5. Digital and Analog video

Flight overview has independent **Digital** and **Analog** panels. Both receive video from USB cameras; the labels describe the rocket-to-ground radio links. In each panel choose **Find**, select a listed USB camera/receiver and press **Start**. Each panel has its own **Stop** and **File...** controls. A selected camera is removed from the other dropdown; active and closing captures also reserve their camera. Select the empty choice to release an unused selection. Stopping or changing one stream leaves the other running. The app uses AVFoundation on Mac, DirectShow on Windows and the existing Video4Linux branch on Linux. A local video file can also be selected. Network-stream URLs are not a dedicated operator input in this version.

The displayed video is fitted into a 960×540 RGB frame with its aspect ratio preserved. Each stream has one FFmpeg input feeding its display and recording. Camera and demo sources are compressed for recording; file inputs use stream-copy for the recorded output. Audio is not captured. Files are stored as approximately 30-second Matroska segments with a CSV time index.

Each video status reports frame age and errors; after two seconds without a recent frame it adds a stale/frozen indication. Stop closes capture but retains the final displayed image with an explicit stopped label. A retained image is not proof of a live stream.

Starting or stopping logging reopens both configured video pipelines so their recording destinations change. A camera reconnect during a recording uses a new segment directory rather than overwriting earlier footage. Video lifecycle work occurs off the GUI thread, but a brief stream interruption is possible.

Replay uses host session timing plus the mission's video offset. A positive offset delays video relative to telemetry; a negative offset advances it. Each stream follows its own recorded starts, segments and gaps. A paused seek decodes one frame per stream. Flight files include both feeds; old single-video sessions appear in Digital. A panel's Stop pauses that replay feed until a seek or playback restart. Compact displays can scroll to the lower flight plots. Camera exposure timestamps are not synchronized with onboard telemetry in this implementation.

6. Antenna assembly and pointing

6.1. Viewing the physical model

The drawing follows the supplied four-leg stand, azimuth turntable, elevation supports and common antenna beam. The grid reflector, long Yagi and enclosed Avenger XR18 share the elevation carrier; mounting centers are attached to the beam concentric with the support pivots. The dish feed arm uses the dish's grid material and lies on the dashed boresight. The Avenger enclosure represents the black unit identified in the supplied photograph; no product photograph is embedded in the renderer.

Drag to orbit the viewing camera, scroll to zoom, and double-click to reset the perspective. Perspective, Front, Side and Top presets are available. Viewing works with serial controls locked and never sends a command. All bearings use the corrected chirality: 0° north, 90° east, 180° south, 270° west.

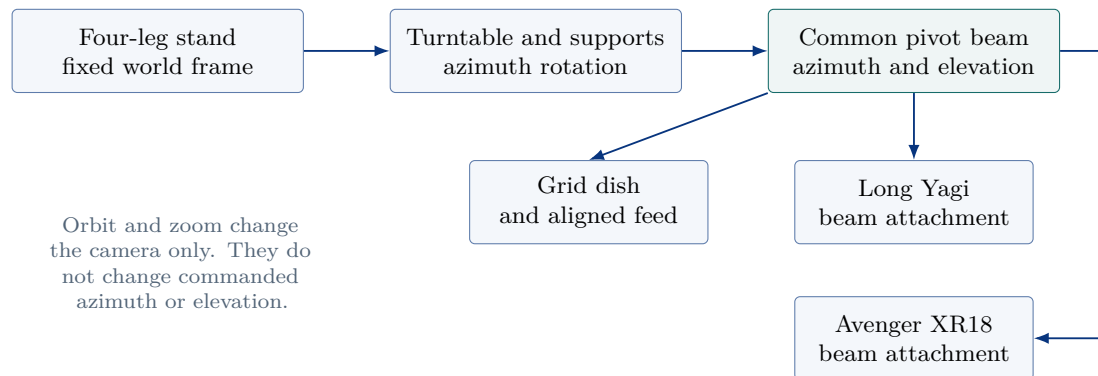


Figure 2: Transform hierarchy of the procedural antenna model. Both outer antennas and the reflector belong to the same moving carrier; this diagram describes attachments, not dimensions or a photographic viewing direction.

6.2. Manual controls and the meaning of ZERO

Enter finite azimuth/elevation values and press **Send**. Editing a text field alone does not move the drawing or transmit. The graphic updates on successful serial dispatch. The legacy manual path does not add mission calibration, software envelope or cable-wrap prerequisites; stored mission envelope fields are not enforcement for these buttons.

UP, DOWN, LEFT and RIGHT send the original firmware direction opcodes in 5-degree increments. These are direction packets, not host-generated absolute targets. Before a known target exists, the app cannot show a reliable absolute pose after a jog.

ZERO sends the firmware's count/target reset at the current physical position. It does not seek a home switch or mechanically return the mount to north/horizontal. The drawing's last-sent label is a command estimate; measured angles and board acknowledgment are unavailable.

6.3. Ground-GPS freeze and automatic tracking

With both boards connected and telemetry polling, press **Send to AntPtr**. The receiver latitude and longitude are frozen at the original five-decimal rounding; its displayed altitude/fix are retained with that snapshot. The legacy calculation uses ground altitude zero and rocket filtered barometric altitude. Mission launch origin, ellipsoid/MSL settings and stored pointer offsets do not replace this convention.

Track live rocket computes targets at up to 5 Hz. It requires fresh LIVE rocket data, usable rocket coordinates and a frozen ground position. The geographic pointing helper rejects targets within 10 m and nearly overhead targets with under 1 m horizontal separation. Stale data, reboot or connection loss stops host tracking; resume explicitly after correcting the condition. **Unfreeze GPS** releases the stored origin and stops tracking.

Stop tracking This stops new targets and removes an unsent queued target. A movement already commanded can continue. The old firmware has no motion-stop opcode, watchdog, measured-pose report or mechanical-homing contract.

7. Rocket commands, power and recovery

7.1. Telemetry subview

The legacy Telemetry subview retains the original values: state, RSSIs, packet age/number, filtered and maximum altitude, velocity, integrated rotations, angle from vertical, temperature, GPS fix/position/heights/precision/satellites, four servo drive/degree readouts and three cell voltages. Cells above 3.7 V use the normal accent, those above 3.5 V the intermediate color, and the remainder the low-value color. These cell cues are separate from the configurable total-battery alert.

7.2. Rocket commands subview

Control	Implemented behavior
Advance State	Requests current decoded state code plus one; retains the original confirmation dialog. The UI does not implement an independent flight-state machine.
Zero P/Y/Roll; Zero Alt; Zero Velo; Zero Servos	Sends the corresponding original opcode. These do not rewrite recorded historical samples.
PD Activate	Sends the legacy activation command. No new controller tuning interface is implemented.
Roll control angle; Airbrakes angle	Sends the entered finite angle as the original float32 payload.
VTX 1, 3, 5 or 8 W	Sends the legacy power-choice index. A selected button represents the request, not a radio acknowledgment.

Ground commands require the ground board in LIVE even when polling is paused. They are simulated in DEMO and disabled in REPLAY. The app does not require fresh radio evidence before every uplink; the separate reception indicator must be interpreted accordingly.

7.3. Power subview

Six converter requests and voltage/current readbacks are shown for 3, 3.3, 5, 7.4, 8.4 and 28 V. Changing an editable request sends the full six-bit mask. The 3.3 V request stays on and is not editable, matching the old UI. Total current and temperature are readouts, not switchable rails. Readback state is derived from voltage proximity to the nominal rail, not from the state of its checkbox.

The BMS table displays Enabled and Triggered rows for SCD, OCD2, OCD1, OCC, COV, CUV and the two reserved bits. It is read-only in the original operator workflow. A BMS command exists at protocol level but is not an additional operator toggle.

7.4. Recovery and pyro subview

Six channels, numbered 0–5, have selections, continuity, armed/fired state and resistance readouts. Select at least one channel before using ARM or FIRE. Both retain the original confirmation behavior and encode the chosen channel mask. The emergency actions PISTON, BP WELLS, TND R DSND R and DEPLOY ALL each retain their own confirmation.

There is no added Disarm button in this subview, although the packet encoder supports the legacy disarm opcode. Continuity and armed/fired displays come from received packets; sending a request alone does not update them as confirmed hardware state. Consult the team's operational procedures for physical recovery system work; this report documents the existing UI rather than a firing sequence.

8. Mission configuration and launch sites

Mission → **Configure** opens a scrollable configuration dialog. Saving validates the entries and applies them; Cancel leaves the current mission unchanged. Stop logging before editing or loading another mission so a recording is not redefined in progress. Fields include the mission name, origin-established flag, location, two altitude datums, legacy height interpretation, canard count, freshness/battery thresholds, launch rail, saved simulation index, random seed and video offset. Appendix A lists every stored field, including legacy fields that are not active UI controls.

8.1. Latitude/longitude, MGRS and Plus Codes

All coordinate conversions are offline:

- **Latitude / longitude:** decimal degrees, latitude -90 to 90 , longitude -180 to 180 .
- **MGRS:** compact or spaced UTM/UPS grid reference. The preview identifies the supplied grid resolution. Conversion uses the grid reference point, not a newly estimated survey position.
- **Plus Code:** a full code resolves to its cell center. A short code requires explicit nearby reference latitude and longitude; locality names are not geocoded.

The preview shows the resolved canonical WGS84 coordinates before Save. Changing only the display format preserves canonical coordinates and avoids round-trip drift. A saved mission keeps the chosen format/code as entry provenance alongside latitude and longitude.

8.2. URRG preset

Select **URRG** from the launch-site preset menu. It sets the exact supplied MGRS string 18TUN2061530290, switches the entry format to MGRS and marks the origin established. Its resolved coordinates are approximately 42.7041678° latitude and -77.1902950° longitude.

The preset supplies coordinates only. It does not infer launch elevation, antenna position, wind or rail settings. Existing elevation values remain for the operator to review. Manual coordinate edits clear the preset name to Custom; changing the display format alone keeps URRG. Mission JSON and flight archives preserve the site selection.

8.3. Mission JSON versus portable flight files

Save mission writes configuration JSON. **Open mission** restores it, clears history/track and deselects the old reference. The JSON contains file paths for a selected rocket and custom motors; it does not embed those files. Loading mission JSON is not the same operation as opening a flight archive and does not itself perform the archive workflow's full source disconnection.

Use **Save flight** for a portable package containing the referenced assets and available data. Keep the distinction in mind when moving work to another machine.

9. Wind and nominal OpenRocket simulation

9.1. Manual and imported wind

The wind table uses height above launch in metres, speed in metres per second and meteorological direction *from* true north in degrees. Add or remove layers and press **Apply wind**. Editing a table cell alone does not update the active mission profile.

Heights must be nonnegative and strictly increasing; there must be 1–200 layers. Speeds must not exceed 150 m/s. A zero-speed direction has no physical effect. Internally each row becomes east and north velocity components:

$$v_E = -s \sin \theta, \quad v_N = -s \cos \theta.$$

Import wind and **Export wind** use JSON with **layers** and a **source** description. The included example shows the exact format. Outside the supplied altitude coverage, the nearest wind layer is held.

9.2. Online weather

Enter an ISO 8601 time with a timezone, such as 2026-09-21T16:00:00Z, and the launch elevation above mean sea level. **Retrieve wind** requests an Open-Meteo pressure-level profile at the mission latitude/longitude. The current implementation selects forecast or historical forecast based on age, rejects requests more than 15 days ahead, and requires an available provider hour within one hour of the selected time. These are app rules, not a guarantee that the provider has every date/location.

Pressure levels are 1000, 925, 850, 700, 500, 300, 200 and 100 hPa. Missing levels and below-launch levels are skipped; remaining layers are sorted. Geopotential height is converted to geometric height, then the launch MSL elevation is subtracted. The provider response, request, endpoint, retrieval time and attribution remain in the mission for offline reuse. Review the resulting coverage. If the launch coordinates change during retrieval, the UI rejects applying that result to the new site.

9.3. Running a reference

1. Configure the launch origin and altitude conventions.
2. Choose the team's `.ork` file. It must contain a saved simulation with a valid active motor configuration.
3. Select the saved simulation index in mission settings; index 0 means the first saved simulation.
4. The Zephyrus MITRT N8406 curve is bundled automatically. For other custom motors, use **Select custom motor files** to provide matching `.eng` or `.rse` curves. Cancel keeps the existing selection.
5. Review rail length, tilt from vertical, true-north heading, wind and seed. Press **Run nominal simulation**. **Cancel simulation** terminates the worker.

A fresh simulation uses the saved vehicle/motor configuration but explicit app-supplied site, rail and wind settings. The bridge disables saved extensions and the fork's global inertia override, uses the ISA atmosphere, zero wind turbulence standard deviation and a fixed seed, and returns the first output branch. The job also records branch count and warnings.

A missing thrust curve is named before simulation; an unpowered configuration is rejected. The supplied Zephyrus test model previously failed because its custom N8406 curve was absent. The bundled curve now matches its required motor digest. The app copies model/motor inputs into each job and records hashes so the run can be traced independently of original input paths.

9.4. Importing a reference and interpreting results

Import reference on Flight overview accepts a CSV and a same-basename JSON manifest. The CSV columns are `time_s`, `east_m`, `north_m`, `up_m`. The manifest declares version 1, ENU, `m,s`, and `origin:[latitude,longitude,ellipsoid_altitude]`; up is launch-relative. A LIVE comparison requires a configured mission origin matching that reference within 10^{-6} degrees and 0.1 m altitude. Synthetic references are rejected for LIVE comparison. Stop logging before replacing the recording's reference.

The saved trajectory is not a flight-control command, and the app does not run an online controlled-vehicle simulation driven by each received packet. For GNC assessment, compare the nominal baseline with actual data while retaining the solver and unavailable-feedback limitations.

10. Recorded Zephyrus demonstration

Choose DEMO explicitly. GS2 is the default; GS1 and GS3 are alternative receivers of the same launch, not consecutive segments. They are never concatenated or deduplicated. Each plays according to its own receive-time deltas, keeping gaps, original UTC, packet numbers and device time.

Receiver	Samples	Duration	Use and limitations
GS1	4,366	274.056 s	Original receiver record, with GPS outliers.
GS2	5,464	330.234 s	Default record; most samples and fewer major GPS corruptions.
GS3	4,939	332.725 s	Contains invalid ground GPS and anomalous rocket GPS/state readings; automatic simulated tracking is unavailable.

The initial cue is five receive-time seconds before the first FLIGHT packet. Use Play/Pause, the timeline, 0.25/0.5/1/2/4 times speed, **Launch -5s** or **Full start**. Stop logging before seeking or changing the receiver log. Pause and speed changes are allowed during a rehearsal recording. At the end the final sample is retained; restarting returns to the launch cue.

The demo track uses horizontal GPS offsets from the first FLIGHT position plus the reported barometric height. It is a local observed track, not a surveyed ellipsoid/MSL trajectory. Missing 3D fixes and positions over 20 km horizontally from its origin are omitted from the plot but kept in raw decoded values. No interpolated packets, touchdown, actuator feedback or corrected state sequence are invented. The records end in Main descent.

Both videos are labeled **TEST PATTERN** (Digital motion pattern and Analog bars); no actual test-launch video was supplied. The original 43 CSV fields remain in each decoded sample's provenance. Raw radio frames were not supplied with these CSVs, so their radio checksums cannot be revalidated. The app verifies the bundled CSV byte hashes on loading.

11. Logging, sessions and portable flight files

11.1. Starting and stopping logging

Start Logging creates a uniquely named session under the application data directory's **sessions** folder. It records accepted samples, events, raw serial bytes, both legacy CSVs and any recorded video. Logging runs independently of UI redraws. A bounded recorder queue reports dropped records and a fault if it overflows; loss is not silently labeled complete.

Stop Logging detaches the active recorder and finishes outstanding video and file work in the background. The event timeline reports completion or error. Closing the app also finalizes its workers. A crash can lose uncommitted data and the active video segment; committed session data may still be replayable.

11.2. Session-folder replay

Use **Open recorded session** to select a session folder. Playback begins paused. Controls provide seek, a 0.1 s step, 0.25/0.5/1/2/4 times speed and manual time alignment. Seeking reconstructs earlier events and up to 120 s of sample history instead of retaining future pointer/time state. The complete session remains indexed on disk even though visible plots are bounded.

Export samples to CSV writes the richer normalized export, including elapsed time and the complete sample JSON. This is distinct from the two original-format CSV files written automatically by the recorder.

Import Zephyrus CSV accepts the audited legacy layout, creates a converted session in a chosen parent directory, retains the original source file and opens it for replay. It preserves available extra columns; unsupported layouts are rejected. Importing decoded CSV cannot recreate raw packet quality or radio bytes.

11.3. Save flight and Open flight

Use **File** → **Save flight** (Command-S on Mac) and **File** → **Open flight** (Command-O). Both are also available in Mission and Sessions. A **.rktflight** archive contains the mission, embedded selected rocket/custom-motor files, current normalized simulation reference, and available session data. It is readable after the original model/motor paths disappear.

State when saving	What is included
Logging active	Consistent committed telemetry/raw/CSV snapshot and finalized video segments. Logging continues. The current unfinished video segment is omitted and the session remains marked as a recording snapshot.
Logging just stopped	Last recording in the current source mode; save waits for pending recorder and video finalization.
REPLAY	Opened session and displayed replay position.
DEMO without a selected recorded session	Complete chosen GS dataset and current demo position; no invented camera footage or raw radio bytes.
LIVE without a recorded session	Only the displayed telemetry buffer, up to 6,000 samples, explicitly identified as a buffer export.
No available telemetry	Planning flight: mission, model/motors and any current simulation reference.

Save again after stopping logging to include the final samples and video segment. A flight archive carries normalized simulation output and metadata, but is not an automatic copy of every file in the original Java job directory; keep that directory for its complete worker log and inputs.

Opening requires logging to be stopped and a running simulation to finish or be cancelled. The archive is validated before switching the current flight. A session-bearing flight opens in paused REPLAY with both transports closed. A planning flight opens in disconnected LIVE. Saved pointer calibration is cleared; no hardware automatically reconnects.

Saving writes a temporary archive and atomically replaces the destination after success; failure leaves an existing destination intact. Loading validates paths, sizes, hashes and supported schemas. A damaged file reports an error instead of silently supplying partial data. Extracted assets are retained under the app's `flights` cache. The file format is versioned but not encrypted or cryptographically authenticated.

11.4. Storage and recovery

The default root is Qt's per-user `AppLocalDataLocation`, under organization `MITRocketTeam` and application `RocketGNCMonitor`. It normally lies under macOS Application Support or Windows Local AppData. Use the paths displayed by the app rather than assuming a fixed home directory. Developers can override it with `-data-dir`.

A session contains `manifest.json`, `session.sqlite`, `raw.bin`, `telemetry.csv`, `telemetry_badpackets.csv`, and optional reference, imported-source CSV and video directories. Do not delete a crashed session's SQLite `-wal` file before recovery: committed records can reside there. Incomplete sessions remain labeled incomplete; a usable database is not a guarantee of complete footage or raw bytes.

12. Alerts, diagnostics and troubleshooting

Diagnostics shows packet counts, the current decoded sample, event messages and recording information. Valid/rejected frames, discarded bytes and inferred sequence gaps describe different conditions. Rejected payloads are isolated from live sample displays and written to the bad-packet CSV when logging.

The LIVE alert manager tracks telemetry stale/absent, low total battery, recording fault/loss and independently identified Digital/Analog video ended/stale. Battery must remain below threshold for 1 s; clearing uses 0.3 V hysteresis. Video alerting has a 1 s dwell after an error or a frame age over 2 s. Acknowledgment records an event and marks the alert; it does not clear the underlying condition. The demo can show low original battery values in readouts without those being a new live-hardware alarm.

Symptom	Checks or explanation
Ground connected, no data	Start Polling. Check that the selected port is the telemetry board and that the rocket/receiver supplies valid frames. USB connection is not radio evidence.
Controls remain locked	Confirm mode LIVE and the relevant board connection. Pointer manual controls need the pointer board; rocket commands/logging need the ground board.
Camera absent or no image	Find cameras again, confirm the receiver enumerates as a camera, grant permission and inspect the video error. Another process may own the device.

Symptom	Checks or explanation
Image visible but no motion	Read frame age and stopped/stale status; the last frame can be retained. DEMO uses a test pattern.
No actual trajectory	Establish launch origin, select the verified height convention and obtain valid 3D GPS. DEMO filters plot-only GPS outliers.
No same-time reference marker	Establish flight time alignment and ensure the displayed time falls within the reference interval.
Missing motor / unusable simulation	Select a saved powered configuration and matching custom curve. The N8406 is automatic. Review the named error and job's <code>worker.log</code> .
Tracking unavailable or stopped	Check both links, polling, frozen ground GPS, target freshness and valid coordinates. Restart deliberately after fixing the problem.
Graphic disagrees with mount	It is a last-sent command model, not feedback. ZERO resets counts at the physical position. Review firmware sign/mechanical reference separately.
Flight save has less video	Active snapshots omit the unfinished segment. Stop logging, wait for completion and save again.
Only a short live flight was saved	Logging was not active; the fallback saves only the bounded display buffer. Start Logging before acquisition for a full capture.
Flight file rejected	Inspect the error for version, checksum, missing asset, path, database or space problems. Keep the original and the session source; do not bypass validation.
Recording incomplete/loss	Inspect recorder error, dropped count and disk availability. Preserve SQLite/WAL and finalized video; neither raw nor CSV can recreate data never received or committed.

13. Keyboard shortcuts and operational limits

Qt preserves the original shortcut strings. On Mac, Ctrl in those strings maps to Command, and Alt maps to Option. The following actions work across workspaces; their enabled state still follows the appropriate connection/mode.

Action	Mac	Windows / Qt notation
Open flight	Command-O	Ctrl+O
Save flight	Command-S	Ctrl+S
Refresh serial ports	Command-R	Ctrl+R
Ground connection toggle	Command-Option-C	Ctrl+Alt+C
Pointer connection toggle	Shift-Command-Option-C	Shift+Ctrl+Alt+C
Polling toggle	Command-Return	Ctrl+Return
Logging toggle	Command-L	Ctrl+L
Pointer UP / DOWN	Command-Up / Down	Ctrl+Up / Down
Pointer RIGHT / LEFT	Command-Right / Left	Ctrl+Right / Left
Pointer ZERO	Command-0	Ctrl+0

Field acceptance still requires the actual boards, USB video receiver and mount, representative data rates, reconnect/power-loss behavior and a launch-duration soak. Three-axis control feedback, a qualified controlled-vehicle model, measured pointer pose and firmware-level stop/acknowledgment are outside the presently verified contract. The current development Mac distribution is ad-hoc signed; production notarization and clean-machine acceptance remain separate release work.

Part II.

Codebase and developer documentation

14. Repository, entry points and source ownership

The application is self-contained under `ui_work/rocket_UI_v0`. Its `src/rocket_gnc_monitor` package is the Python application; `simulation_bridge/src/RocketBridge.java` is the Java integration layer. The old PyQt5 ground-station folder is a behavioral reference, not a runtime dependency. Production builds use bundled resources and do not need the original telemetry, OpenRocket source or font-system paths.

The console entry point `rocket-gnc-monitor` and `python -m rocket_gnc_monitor` both call `__main__.main`. The packaged launcher calls that same function. It parses arguments, registers Lucida Grande, names the Qt organization/application, resolves the per-user data directory, constructs `MainWindow` and enters the Qt event loop. The window owns one `Controller`; its close event invokes controller shutdown.

```
rocket_UI_v0/
src/rocket_gnc_monitor/  # application package
simulation_bridge/src/   # Java bridge source
resources/               # icon, fonts, demos, motors, examples
docs/                   # in-app Markdown help and dependency notices
tests/                  # unit, Qt, serial and engine integration tests
tests/fixtures/         # captured legacy contract and small .ork fixture
scripts/                # setup, build and package verification
packaging/              # PyInstaller spec, launcher, entitlements
vendor/                 # staged engine, bridge, JRE, build manifests
build/                  # generated jobs, logs and validation artifacts
dist/                   # distributable app/DMG or folder/ZIP
Makefile                # shared native build entry points
Makefile.macos / Makefile.windows
pyproject.toml / uv.lock / requirements-build.txt
```

14.1. Module map

Paths in this report link to the reviewed source relative to the existing repository layout. Use the named symbols rather than fixed line numbers as maintenance anchors.

Module	Responsibility and main seams
src/rocket_gnc_monitor/__main__.py	CLI/startup; Qt data path and fonts; startup, demo, flight and engine smoke modes.
src/rocket_gnc_monitor/controller.py	State owner and orchestration: connection generations, polling, command dispatch, tracking, mode changes, recording, replay, file jobs, alerts and shutdown.
src/rocket_gnc_monitor/ui.py	<code>MainWindow</code> , <code>MissionDialog</code> , workspace construction, shortcuts, file dialogs, result presentation, enabled states and plots.
src/rocket_gnc_monitor/rocket_panel.py	Complete legacy rocket Telemetry/Commands/Power/Recovery subviews and retained confirmations.
src/rocket_gnc_monitor/widgets.py	Themes, <code>AttitudeView</code> , <code>MountView</code> , local geometry, material colors and camera/pose transforms.
src/rocket_gnc_monitor/fonts.py	Private font registration and global Qt font choice.
src/rocket_gnc_monitor/domain.py	<code>Mission</code> , <code>Sample</code> , finite checks, geographic transforms, pointing and wind validation. Contains a synthetic test-data helper.
src/rocket_gnc_monitor/location.py	Qt-independent coordinate conversion and <code>Location</code> .
src/rocket_gnc_monitor/location_ui.py	<code>LaunchLocation</code> widget; format/preset state, preview and edit-versus-display distinction.
src/rocket_gnc_monitor/devices.py	Port identity inventory and bounded independent <code>SerialWorker</code> threads.
src/rocket_gnc_monitor/protocol.py	Incremental <code>ZephyrusDecoder</code> , pointer packet builder and legacy route-validation helper.

Module	Responsibility and main seams
<code>src/rocket_gnc_monitor/zephyrus.py</code>	Rocket command opcodes/packet building and canonical 43-column legacy CSV adapter.
<code>src/rocket_gnc_monitor/legacy_sample.py</code>	Restricted AST-based CSV literal parsing and full row-to-sample conversion.
<code>src/rocket_gnc_monitor/legacy.py</code>	Validated legacy CSV-to-session conversion, provenance and incomplete-import handling.
<code>src/rocket_gnc_monitor/demo.py</code>	Bundled CSV integrity, launch cue, independent receiver clocks and plot-only ENU frame.
<code>src/rocket_gnc_monitor/recording.py</code>	Asynchronous SessionRecorder , indexed read-only SessionReader , raw capture CRC reader.
<code>src/rocket_gnc_monitor/flight.py</code>	Portable archive snapshot, atomic save, validated extraction and LoadedFlight .
<code>src/rocket_gnc_monitor/media.py</code>	FFmpeg location, camera discovery, process arguments, frame queue, recording and orderly stop.
<code>src/rocket_gnc_monitor/trajectory.py</code>	Trajectory , online wind normalization, motor resources and isolated SimulationJob .
<code>src/rocket_gnc_monitor/__init__.py</code>	Package version identifier.
<code>simulation_bridge/src/RocketBridge.java</code>	Fork-specific OpenRocket loading, motor resolution, nominal options, trajectory/error publication and private preferences.

14.2. Dependencies and resource resolution

Python is the main application language because Qt, serial I/O, numerical plots and filesystem orchestration fit one runtime. Java is isolated to the existing team OpenRocket fork. The GUI uses PySide6 Essentials and PyQtGraph with NumPy; serial transport uses pyserial, video uses imageio-ffmpeg's native FFmpeg, and location conversion uses mgrs and `openlocationcode`.

The project permits Python 3.12 through 3.14 in metadata; the build guidance and tested environments center on 3.12–3.13. Dependency ranges live in `pyproject.toml`; `uv.lock` and `requirements-build.txt` pin build environments. Do not treat the range bounds as a tested matrix.

Source execution resolves resources from the repository root. Frozen execution uses PyInstaller's `sys._MEIPASS`. FFmpeg is located through its wrapper; Java/engine/bridge are resolved under `vendor`. A wheel or editable Python package alone is not the complete production distribution: package the resource tree and native runtimes through the supplied build workflow.

15. Runtime architecture and state ownership

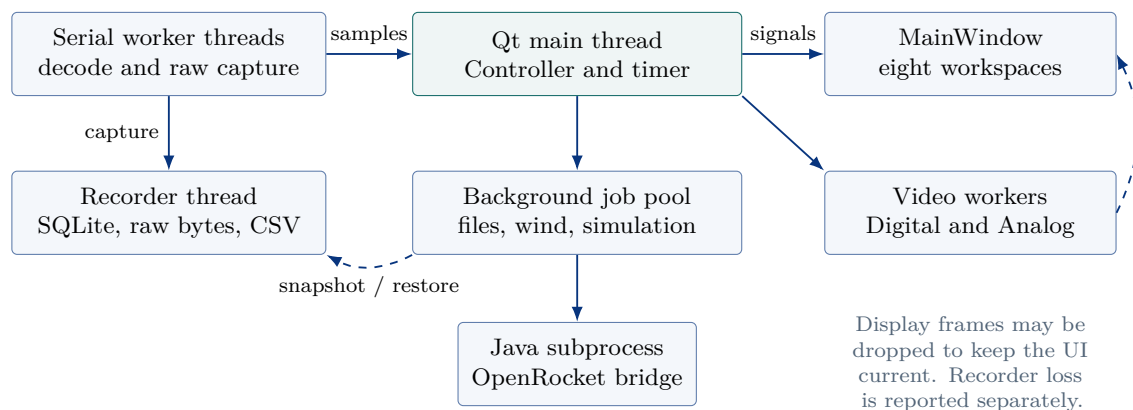


Figure 3: Runtime ownership. Hardware reads, file compression and engine execution do not belong in Qt paint or input callbacks. LIVE accepted samples enter the recorder before the display queue.

15.1. Concurrency model

Qt widgets and controller state are updated on the main thread. Serial workers own their ports and decoders. The recorder has its own thread and SQLite connection. General background jobs use a four-worker executor; video open/close operations use one single-worker executor per stream to serialize its FFmpeg lifecycle changes independently. Each decoder thread drains its FFmpeg frames, with a separate stderr drain.

The controller's 30 ms timer drains up to 2,000 queued messages per tick, advances DEMO/REPLAY, dispatches tracking when due, takes the most recent frame from each video stream, collects completed futures and updates alerts. Playback delta is capped at 0.3 s per tick, so a long blocked GUI does not produce an arbitrarily large playback jump. This desktop scheduler is not a hard real-time control loop.

Buffer / cadence	Bound	Consequence
Controller messages	16,000 messages	UI overflow increments ui_drops ; stale generations are ignored.
Displayed history and track	6,000 each	Bounded UI memory; not the full session archive.
Plot time-series tail	1,200 samples	Altitude/rate/rotation plots use a shorter recent tail.
UI refresh	About 10 Hz	refresh throttles updates to 0.1 s. Some tables refresh every 0.5 s.
Telemetry command queue	64 commands	Bounded; each command expires after 0.5 s.
Pointer command queue	1 command	No silent overwrite; a pending dispatch prevents another ordinary target.
Tracking	At most 5 Hz	New target only if the preceding one is not pending.
Recorder queue	16,000 entries	LIVE overflow records loss/fault. File import can wait rather than drop rows.
Recorder commit	About every 0.25 s	Raw/CSV are flushed and fsynced before committing matching indexes/state.
Video display queue	2 frames per stream	Old display frames are dropped for freshness, independently of FFmpeg recording.
Replay history reconstruction	Previous 120 s	Earlier samples remain on disk and can be sought explicitly.

In LIVE, `Controller.enqueue` sends an accepted sample to the recorder before putting it into the UI queue. Raw callbacks also reach the recorder directly. Thus a dropped UI message need not mean the same sample was lost from disk. Conversely, recorder overflow is an independent data-loss condition. Do not merge these counters or move capture entirely behind UI repainting.

15.2. Connection generations and asynchronous results

A new serial connection receives a generation token. Worker messages are used only if the current role still owns that generation and the mode is LIVE. This prevents a late error/sample from an old worker from altering a new link. Each video stream uses a separate generation and lifecycle executor to prevent an old open result from replacing a newer selection. Camera reservations cover pending opens and closes as well as running capture.

General jobs report through the controller's future list and `task_done` signal. Weather/reference/simulation jobs retain source-mode identity; completed work from a different mode is not silently installed. Simulation results are also compared against the relevant mission inputs. The UI performs additional weather site checks and reference-origin validation.

Flight operations use `flight_busy`. Saving captures copies of mutable mission/reference/history state before background I/O. Opening stores the connection generation and mission snapshot; if source/settings change while loading, the result is not applied over the newer context.

15.3. Mode transitions

`switch_mode` validates the target and verifies a requested demo resource before changing the current source. It stops logging, closes replay/serial/video resources, cancels host tracking, clears histories and command state, resets

time and sample counters, clears pointer calibration and installs the new mode. DEMO saves the prior reference for restoration when leaving.

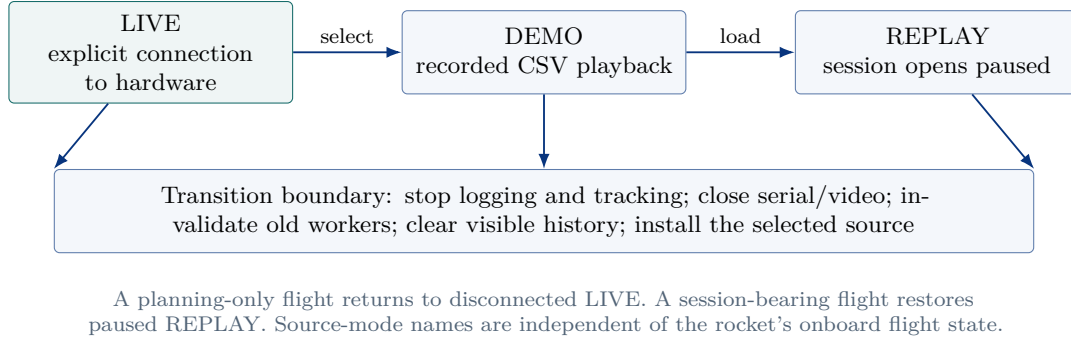


Figure 4: Mode isolation. Other mode selections use the same transition boundary; the upper arrows illustrate common operator paths rather than the only permitted transitions.

`open_replay` validates the session's mission/reference before replacing the current source, then enters REPLAY, restores the manifest, seeks to zero and stays paused. `apply_flight` restores an extracted flight's mission, reference and saved position; a planning-only file forces a clean LIVE state. Neither path restores a physical connection.

15.4. Lifecycle and shutdown

`Controller.shutdown` is idempotent. It stops the Qt timer, stops serial workers, closes video and waits for both video executors, cancels the active simulation, closes recorder/reader resources and waits for background work. Pending recording-close work is allowed to finish. Keep this ownership order when introducing a new worker so it cannot write to an already closed resource.

16. Data contracts, units and geographic frames

16.1. Mission and Sample

`Mission` is a dataclass of persisted configuration. `validate` checks coordinate/numeric ranges, supported location/height choices, actuator count, seed/index, motor-path list and wind profile. `save` writes schema version 1 through temporary-file replacement. `load` accepts version 1, filters to known fields, uses dataclass defaults for absent fields and always clears `pointer_calibrated`. There is no general multi-version migration framework.

`Sample` is a dataclass representing one accepted or imported sample. It contains device/receive times, source, packet number, navigation/kinematic values, phase, actuator dictionary and extensible details. Optional missing measurements use `None`; they are not replaced by zero. The live decoder converts nonfinite floating sensor values to unavailable values. JSON output rejects NaN. Appendices A and B give all fields.

`details` carries original legacy fields, provenance, boot index and quality annotations. Extend it conservatively: legacy rendering, CSV export, record/replay and archive validation all consume samples. New actuator entries should use explicit `demand`, `measured` and/or `drive` keys with declared units, rather than renaming the four unknown legacy channels.

16.2. WGS84 and local ENU

`ecef` uses WGS84 semi-major axis $a = 6378137$ m and eccentricity squared $e^2 = 6.69437999014 \times 10^{-3}$. For latitude ϕ , longitude λ , ellipsoid height h :

$$N(\phi) = \frac{a}{\sqrt{1 - e^2 \sin^2 \phi}}, \quad \mathbf{x} = \begin{bmatrix} (N + h) \cos \phi \cos \lambda \\ (N + h) \cos \phi \sin \lambda \\ (N(1 - e^2) + h) \sin \phi \end{bmatrix}.$$

`to_enu` subtracts the origin ECEF vector and applies the standard local east/north/up rotation at that origin. The plotting contract is (E, N, U) , with $E \times N = U$. `pointing` converts the local displacement to

$$\text{az} = \text{atan2}(E, N) \bmod 360^\circ, \quad \text{el} = \text{atan2}(U, \sqrt{E^2 + N^2}).$$

The helper rejects the near-origin and nearly overhead cases described in Part I. The live legacy pointer caller deliberately chooses its own origin/height convention; it does not call the mission trajectory conversion as a substitute.

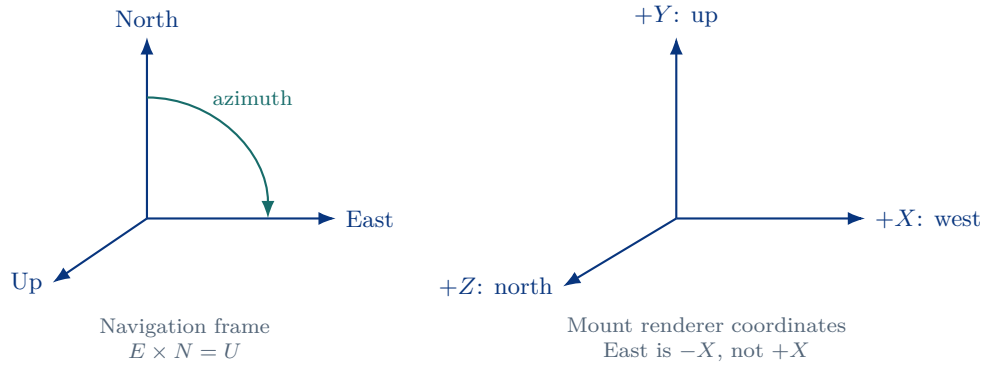


Figure 5: The navigation and renderer bases are different but both preserve the intended chirality. The diagram is a projected axis sketch; up on the page is not a universal physical direction.

16.3. Time normalization and reboot behavior

The decoder extends a 32-bit millisecond rollover when a backward jump exceeds 2^{31} . A smaller backward jump starts another boot epoch, resets rollover offset and clears sequence-gap history. Sequence gaps use modulo-65536 differences and count missing values only for forward deltas between 2 and 32767. The controller uses boot-index changes to invalidate alignment and old history.

For recording, elapsed time is the host monotonic receive time minus recorder start, rounded to microseconds. Replay is driven by this elapsed column, not by pretending onboard uptime began at zero. Events restore alignment and last sent pointer state. Imported CSV uses original receive timestamps relative to its first row. DEMO uses its receiver's own timestamps; it does not merge the three clocks.

17. Serial and protocol implementation

17.1. Port ownership and transport rules

`ports` returns device, description, serial number and an identity derived from VID/PID/serial when possible. `Controller.connect` enforces LIVE, nonempty selection and distinct board ports, then constructs the worker. Connecting does not start telemetry polling or transmit a setup command.

The worker opens pyserial with the role-specific read timeout and a 0.2 s write timeout. Queued writes call `cancel_read` where available to wake the worker. Only the telemetry role has the polling gate. Pointer RX is logged as bytes and hex status, not decoded as authoritative measured angles. Each write checks expiration, records the actually written prefix, and emits `sent` only if the complete payload was written. Errors close the port; there is no retry loop.

17.2. Telemetry framing and checksum

A telemetry frame is 144 bytes:

```
offset 0..1      AB AB synchronizer
offset 2..129    128-byte Zephyrus payload P
offset 130..143 14-byte ground receiver trailer G
```

```
P[127] = sum(P[0:127]) modulo 256
```

The incremental parser retains partial frames and a trailing potential sync byte. After a bad checksum it advances one byte and rescans, so an overlapping valid synchronizer is not thrown away. It accepts concatenated reads and reports accepted, rejected, discarded-byte and inferred-gap counters. The additive checksum covers the payload only, is weak error detection and does not authenticate a sender.

Payload offsets are relative to $P[0]$; multibyte fields below are little endian unless stated otherwise.

Bytes	Field	Decode / interpretation
0–1	Pyro continuity	Six two-bit states from the packed 16-bit value.
2, 3	Armed, fired	Channel bit masks; channels 0–5.
4–9	Pyro resistance	Six unsigned bytes divided by 10.
10–15	Servo drives	Four packed 12-bit values, retained as legacy drives.
16–24	Acceleration XYZ	Three signed 24-bit integers divided by 12800 and multiplied by 9.80665 ; m/s^2 .
25–30	Gyro XYZ	Signed int16 values times 0.03051757812 ; legacy Y sign inverted; degrees/s.
31	Rocket GPS fix	Raw fix code; 3D trajectory requires at least 3.
32–39	Latitude, longitude	Two signed int32 values times 10^{-7} degrees. Invalid bounds clear coordinates/fix.
40–43	GPS height	Float32 metres; interpretation is mission-dependent.
44–51	Horizontal/vertical accuracy	Two uint32 values divided by 1000.
52	Satellites	Unsigned byte.
53–58	Raw pressure, temperature	Two 24-bit raw words; temperature calibration is in the legacy adapter.
59–62	Filtered altitude	Float32 metres.
63	State	0 ground testing, 1 preflight, 2 flight, 3 post-apogee, 4 main, 5 end.
64–75	Integrated rotations	Three float32 values at 64, 68, 72; rendered roll/pitch/yaw.
76–79	Maximum heights	Barometer then GPS uint16.
80–83	Onboard clock	uint32 milliseconds uptime.
84–85	Packet sequence	uint16 modulo counter.
86	Onboard RSSI	Signed byte minus 99.
87–92	Cell voltages	Three signed int16 values divided by 1000.
93–94	Total current	Signed int16 divided by -1000 .
95–97	BMS	Temperature byte divided by 2; protection status and enabled bits.
98–109	Six rail voltages	Signed int16 times 0.0016 .
110–121	Six rail currents	Signed int16 times 0.000625 .
122–125	Velocity	Float32 metres/s.
126	Unmapped byte	Not interpreted by the present decoder.
127	Payload checksum	Sum modulo 256 of bytes 0–126.

Trailer G is RSSI at 0 (signed byte minus 99), fix at 1, signed int32 latitude at 2, longitude at 6 (both 10^{-7} degrees), and signed int32 height at 10. The legacy adapter divides receiver height by 1000 even though the audited firmware supplies metres. Preserve this discrepancy explicitly; the legacy pointer origin instead uses height zero.

17.3. Pointer command packets

byte 0	AA
byte 1	opcode
bytes 2..5	azimuth float32 LE (opcode 0), otherwise zero
bytes 6..9	elevation float32 LE (opcode 0), otherwise zero
byte 10	sum(bytes 1..9) modulo 256

Opcodes are 0 absolute, 1 UP, 2 DOWN, 3 LEFT, 4 RIGHT and 5 ZERO. The legacy firmware negates internal azimuth and uses a shortest-route policy; the host preserves original direction opcodes rather than reinterpreting them. Absolute finite values are serialized without invoking the old `validate_route` helper.

`validate_route` and mission envelope/calibration fields remain in source for compatibility and tests of that helper. They are not called by the current manual control/tracking dispatch path. Do not document them as an active mechanical interlock or silently enable them when modifying the legacy UI.

17.4. Rocket uplink packets

```
byte 0      AA
bytes 1..12  zero/reserved, except:
             byte 12 integer argument, or
             bytes 9..12 float32 LE angle
byte 13      command opcode
bytes 14..15 sum(bytes 1..13) modulo 65536, BIG endian
```

The mixed endian checksum is deliberate and covered by captured vectors.

Opcode	Encoder command	Argument
01 / 06 / 07	Arm / fire / disarm pyros	Nonempty bitmask of channels 0–5.
02	Advance state	Requested state code; UI uses current + 1.
03 / 05	Airbrakes / roll servo	Finite float32 angle.
04	PD activate	No parameter.
08 / 09	Zero servos / P-Y-roll	No parameter; zero-roll API aliases 09.
0A / 0B	Zero altitude / velocity	No parameter.
10 / 11 / 12 / 13	Piston / BP wells / tender descender / all	No parameter.
14	VTX power	Index 0–3 for 1/3/5/8 W.
15	Converter requests	Six ordered Boolean request bits.
16	BMS protections	Boolean encoded 0/1; API exists, operator status is read-only.

`rocket_packet` validates command/value types. UI confirmation and transport authorization remain separate layers: construct bytes without side effects, confirm the original sensitive actions in `RocketPanel`, then dispatch through the controller.

18. Recording, replay and portable archive internals

18.1. Session writer and durability

`SessionRecorder` creates `session_YYYYMMDD_HHMMSS<suffix>` and writes an initial manifest with `complete:false`. It enqueues sample dictionaries, events and role-tagged raw byte chunks. One writer thread owns all session output. SQLite uses WAL and `synchronous=FULL`; ordinary commits occur about every 0.25 s and at closure.

```
CREATE TABLE samples(
  id INTEGER PRIMARY KEY, elapsed REAL, t REAL, data TEXT);
CREATE INDEX sample_time ON samples(elapsed);
CREATE TABLE events(
  id INTEGER PRIMARY KEY, elapsed REAL, data TEXT);
CREATE INDEX event_time ON events(elapsed);
CREATE TABLE raw_index(
  id INTEGER PRIMARY KEY, elapsed REAL, role TEXT,
  offset INTEGER, length INTEGER);
CREATE TABLE snapshot_state(
  id INTEGER PRIMARY KEY CHECK(id=1), data TEXT);
```

`samples.data` is JSON from `Sample.to_dict`; `events.data` is `{"name":..., "data":...}`. A raw index offset points to the chunk header in `raw.bin`; its length is the payload length, not the whole chunk length.

Before a database commit, raw bytes and both CSV outputs are flushed and fsynced. The same SQLite transaction stores a `snapshot_state` row with matching raw/CSV byte lengths, manifest snapshot, duration, error and dropped count. A successful final close checkpoints WAL and updates the manifest with completion, duration and error/loss information. Queue overflow marks loss; a close timeout or writer exception leaves an incomplete/error condition.

The recorder's own decoder watches raw telemetry RX to produce the rejected packet CSV. Bad packets do not become `samples` rows. Good and bad CSV field order comes from `CSV_FIELDS`; the adapter retains the original padding and calibrations while fixing the documented old bad-packet-only inconsistencies.

18.2. Raw capture format

Each raw chunk has a 25-byte little-endian header with `struct.Struct("<4sddBI")`:

Offset	Type	Meaning
0–3	4 bytes	Magic RGM1.
4–11	float64	Host session elapsed seconds.
12–19	float64	UTC Unix seconds.
20	uint8	Role: telemetry RX 1, pointer RX 2, pointer TX 3, telemetry/rocket uplink TX 4.
21–24	uint32	Payload byte count.
25 onward	Byte payload	Exactly the declared number of raw serial bytes.
Following payload	uint32 LE	CRC32 of header plus payload.

`read_raw` yields complete CRC-checked chunks and rejects wrong magic, payloads over 4 MiB, truncated headers/-payloads and CRC mismatch. This capture CRC protects stored chunks; it does not add authentication to the radio or repair the unchecksummed receiver trailer.

18.3. Replay reader

`SessionReader` opens SQLite read-only, reads manifest version 1, gets sample count and maximum sample elapsed time, and runs `PRAGMA quick_check`. `at(t)` returns the latest sample at or before t ; `between(a,b)` returns $a < elapsed \leq b$, ordered by elapsed and id, with a default 20,000-row limit. `events(t,begin)` uses the same temporal selection convention.

`seek` clamps to the session duration, clears future state, resets initial time alignment, applies prior events and reconstructs a 120 s history window. Received monotonic times are refreshed for display. Normal play advances the cursor at the selected speed, applies events and samples in the traversed interval, and pauses at the end. Events currently restore flight-zero and sent-pointer state; do not assume every console string is executable replay logic.

18.4. Flight archive layout

The portable extension is `.rktflight`; its contents are ZIP64 with a `flight.json` inventory. Files normally use DEFLATE level 3, while MKV is stored without another compression layer.

```

flight.json           # format/version, source/scope, position, inventory
mission.json         # version 1; asset paths relative to this archive
assets/rocket.ork    # optional selected model
assets/motor_000.eng  # optional selected curves (.eng or .rse)
reference.csv        # optional current normalized reference
reference.json       # only if a session was exported
session/session.sqlite # standalone staged database, no -wal/-shm
session/raw.bin
session/telemetry.csv
session/telemetry_badpackets.csv
session/legacy-source.csv # if available from legacy import
session/reference.*   # if present
session/video*/segments.csv
session/video*/segment_*.mkv

```

The inventory records byte count and SHA-256 for every payload file; it does not recursively checksum itself. Top-level keys include `format:"rocket-gnc-flight"`, `schema_version:1`, app version, save UTC, source mode, export scope, sample count, optional session path, playback position, flight-zero/alignment and video-scope description.

Selected user motor files are embedded under generated portable names. The app's own standard/bundled motor library is part of the runtime distribution; it is not duplicated in every flight. Original job hashes and request

metadata may remain in the reference manifest as provenance; the live model/motor paths are rebound to extracted assets.

18.5. Active snapshot consistency

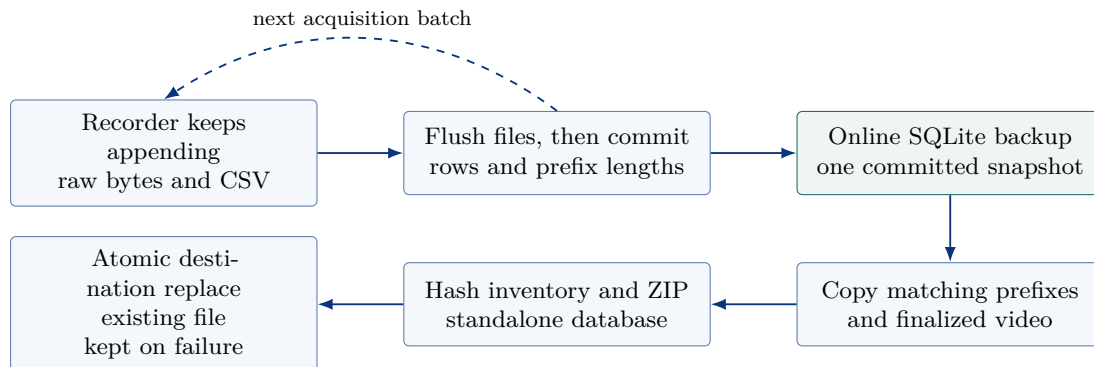


Figure 6: Save during logging. The snapshot boundary is a committed database transaction with matching file lengths, not an uncontrolled copy of an active SQLite file.

`snapshot_session` uses SQLite’s online backup API to capture a committed database without pausing the serial writer. It then reads the copied `snapshot_state` to find exactly which raw/CSV prefixes belong to that database version. Later appends to the source do not change those prefixes. An early save waits briefly for the first committed state. Older stable sessions without snapshot state use their available files and retain their completion status.

Video is copied from finalized entries in the segment CSV; a partial trailing index line is ignored for active capture, and segments beyond the snapshot duration are omitted. After logging stops, the controller waits for pending video/recorder closure before saving. No unfinished camera file is claimed as complete footage.

The staged database is checkpointed and changed to DELETE journal mode once all staging readers close. Only this owned staging copy is changed; the active source database stays in WAL mode. This avoids shipping platform-specific shared-memory state. A temporary archive is fsynced and replaced atomically at the requested destination.

18.6. Archive validation and application

`load_flight` extracts into a new UUID directory under the owned flight cache. It rejects unsupported versions, encryption, linked/directory/duplicate case-folded entries, absolute/traversal/backslash/colon/control-character paths, mismatched inventories, declared sizes and content hashes. Loading limits are 64 GiB of expanded entries, 100,000 entries and 16 MiB per metadata object; free space is checked before extraction.

Mission asset paths must resolve to included assets. Mission/reference data, SQLite integrity, sample/event structure and video segment indexes are validated before the controller applies a `LoadedFlight`. On failure, the new extraction directory is removed. Parsing uses JSON and ordinary media/model formats, not pickle or evaluation of archive-supplied Python.

The archive inventory is corruption detection, not a signature or trust identity. A deliberately altered payload with a recomputed hash is still subject to schema checks, but is not proven to have come from the team. File limits and validation should remain aligned when evolving save and load behavior.

18.7. Read-only analysis example

Run analysis with the app's development environment so its domain contracts match the recording:

```
from pathlib import Path
from rocket_gnc_monitor.recording import SessionReader, read_raw

session = Path("/absolute/path/to/session_folder")
reader = SessionReader(session)
try:
    print(reader.count, reader.duration, reader.manifest["complete"])
    for elapsed, sample in reader.between(-1, min(10, reader.duration)):
        print(elapsed, sample.t, sample.altitude)
    reader.export_csv(session.parent / "analysis-export.csv")
finally:
    reader.close()

for elapsed, utc, role, payload in read_raw(session / "raw.bin"):
    print(elapsed, role, len(payload))
```

Do not feed replay events to a serial worker. Reading archive/session data is separate from authorizing hardware commands.

19. Legacy import and demo provenance

`numeric_list` uses a small AST visitor. It accepts numeric/Boolean literals, bounded list/tuple forms, unary signs and known one-argument NumPy numeric wrappers, plus known Boolean names. It rejects arbitrary calls and attributes; it never invokes `eval`. Numeric-array text is limited to 4,096 characters and arrays to 32 elements.

`import_legacy` rejects files over 200 MB and known bad-packet filenames, requires the audited core headers, checks monotonic receive timestamps and refuses empty datasets. File import can block on the recorder queue to preserve all rows. It stores source bytes and hash, records observed imported flight transitions, and explicitly marks raw packets unavailable. A failed import retains an incomplete/error manifest rather than presenting a completed session.

`DemoFlight` verifies the uncompressed SHA-256 and row count from `resources/demo/manifest.json`; gzip is lossless packaging. It computes receiver-relative times, locates the first FLIGHT row and exposes a launch cue and indexed samples. Each sample contains the original row/UTC, station, source file/hash and local-frame note. Plot-only rejection of invalid/far GPS does not erase the original values.

The functions `domain.demo_sample` and `Trajectory.demo` remain useful synthetic fixtures and examples. They do not drive the current operator DEMO telemetry. Avoid reconnecting these helpers to the source selector when adding features; the user-visible default is the actual GS2 log.

20. Video implementation and synchronization

`camera_devices` invokes bundled FFmpeg to enumerate AVFoundation or DirectShow devices (and Linux video devices in its existing branch). `VideoWorker.arguments` builds an argument list, without a shell, for camera, file or test-pattern input. It maps only video and disables audio.

The display pipe is `rgb24` at 960×540 . The decoder reads exactly one full frame before queueing it; incomplete EOF frames are discarded. The two-frame queue trades display history for low latency. Recording is a separate FFmpeg output from the same input: MPEG-4 for live/test sources, stream-copy for local files, Matroska segments and `segments.csv`. The display scaling does not imply the stored camera stream has the same dimensions.

`Controller.video_streams` maps `digital` and `analog` to `VideoStream` state: worker, backend, configuration, generation, reserved cameras, replay key/pause state, lifecycle future and open error. `video_frame` carries the stream identifier and bytes; `video_reset` clears only that panel. The legacy `video/video_config` properties and `frame` signal remain first-stream aliases.

`start_video(..., stream="digital")` increments that stream's token and schedules work on its single-worker executor. Duplicate camera sources are rejected before altering either stream, including a camera whose prior handle is still closing. `stop_video(stream)` closes one feed; omitting the stream closes both. The UI filters camera selections and uses `serial_device_key` to exclude active board ports and macOS aliases from the other serial

dropdown. **stop** first writes **q** and continues draining output so FFMpeg can finish the trailer/index, then uses bounded terminate/kill fallback. Keep stderr draining independent so logs cannot block frame acquisition.

A **Video recording started** event records session elapsed time, source kind, video directory and **stream**. Every recording start allocates a unique **video_<stream>_<uuid>** directory, even when opens are queued rapidly. Stopping logging waits for both lifecycle futures before closing the recorder, including a feed manually stopped just beforehand. Old events without a stream identifier map to Digital. Replay finds the eligible recording segment at

$$t_{\text{video}} = t_{\text{replay}} - t_{\text{start}} - \Delta_{\text{offset}}.$$

It chooses each stream's segment whose interval contains that time and seeks relative to its start. A manually stopped replay feed stays stopped until an explicit seek or playback restart. Playing checks segment transitions every 0.2 s; a replay key avoids reopening an unchanged segment. The reader's media events and segment indexes are cached so these checks do not repeatedly scan the event log. Gaps stop only the affected feed. Positive offset therefore delays video. Paused replay uses **single_frame**; playback uses the requested speed. This is host timing alignment, not camera exposure or hardware clock synchronization.

21. Weather and OpenRocket bridge contracts

21.1. Weather normalization

weather_profile parses timezone-aware ISO timestamps and normalizes to UTC. It caps response bytes at 4,000,000 and uses a 15 s network timeout. The nearest returned hour must lie within 3,600 s. It transforms geopotential height H to geometric height using $R = 6356766$ m:

$$h_{\text{AGL}} = \frac{RH}{R - H} - h_{\text{launch,MSL}}.$$

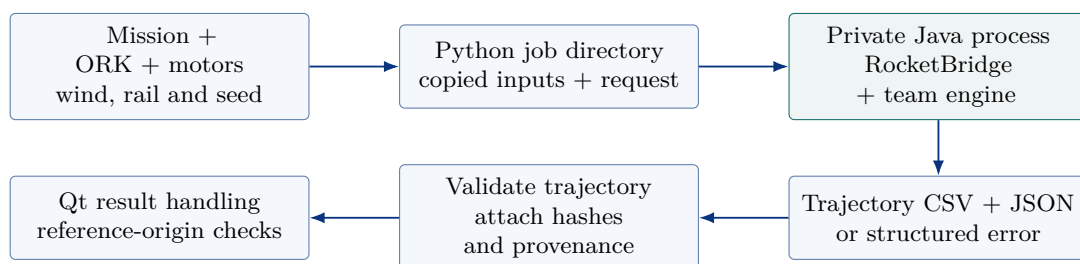
The returned object contains **layers**, **raw**, **source**, **request**, **endpoint** and **retrieved_utc**. Network work stays in the job pool. The mission stores that object in **weather_raw**. Manual profiles use exactly the same validated east/north layer representation.

21.2. Trajectory contract

A trajectory is an $N \times 4$ NumPy array (t, E, N, U) plus JSON metadata. It requires 2–200,000 rows, finite values and strictly increasing time. Loading caps the CSV at 40 MB. The manifest must declare version 1, ENU and **m,s**; a nonsynthetic reference needs a valid three-value geographic origin. **at(t)** interpolates each spatial coordinate linearly within coverage and returns unavailable outside it. Save produces CSV and a same-basename JSON.

21.3. Simulation job boundary

The Python job copies the model and all explicit/bundled custom curves, writes **request.json**, launches the private Java executable and captures its output in **worker.log**. The request contains schema version, copied asset paths, bundled motor count, simulation index, launch latitude/longitude, MSL altitude, ellipsoid origin altitude, rail settings, seed, wind layers, output directory and nominal/inertia flags.



Cancellation and worker limits do not stop telemetry acquisition. Simulation results remain nominal and are never translated directly into physical control commands.

Figure 7: The simulation boundary uses files and a subprocess, keeping the JVM outside the Qt process.

Limits are 40 MB for the model file, 100 MB expanded ORK archive contents, 10 MB per motor curve, 120 s of worker wall time and 20 MB of worker log. Cancellation sets an event and terminates the child; timeout/oversized-log paths also stop it. Result collection checks return status, prefers a structured `error.json` message and otherwise includes a bounded log tail.

The bridge runs headless with private in-memory preferences. It loads the team's motor database, adds hash-verified bundled curves and explicit custom curves, loads the ORK document and verifies the selected saved configuration. A missing motor warning becomes a named error before simulation. A configuration without an active motor is rejected.

The fork-specific nominal setup disables saved simulation extensions and its global inertia override. It sets fixed rail direction rather than *launch into wind*, a 0.05 s step setting, a 600 s simulation maximum, ISA atmosphere, multilevel AGL wind and zero turbulence standard deviation. The Java wind direction is *toward*, $\text{atan2}(v_E, v_N)$, in contrast to the operator's meteorological *from* input.

The first branch contributes time, X/east, Y/north and altitude/up. Nonfinite/nonincreasing rows are skipped; fewer than two rows is an error. Output metadata records branch count, motors and their engine digests, saved simulation name, warnings, nominal qualification flags and solver description. Python adds engine/model/motor hashes and the exact request/job directory. The bridge exits explicitly after success so engine loader threads cannot keep the worker alive.

21.4. Reproducibility and interpretation

The bundled MITRT N8406 curve is `resources/motors/TestLaunch_v14.eng`. Its engine digest is `057a820844b93fe3307401623a92d4c6`. Byte integrity and model-digest matching are separate checks: identical naming alone is insufficient.

A fixed seed and copied inputs support reproducibility on the pinned engine. Results do not automatically transfer to a different OpenRocket binary or qualified controlled-flight model. The package retains the complete team engine; a random upstream JAR is not a drop-in replacement for fork-specific fields. The exact source revision corresponding to the supplied engine binary remains a release provenance item in the existing build metadata.

22. UI, font and antenna graphics maintenance

22.1. UI responsibilities

Workspace builders in `MainWindow` construct widgets; event handlers call the controller or submit background jobs; `refresh` presents snapshots. `refresh_connections` and `RocketPanel.set_controls_enabled` express the existing board/mode rules. `guard` reports exceptions in the event area, status bar and warning dialog. Do not swallow a rejected operation by leaving a success-looking UI state.

`install_shortcuts` creates each `QAction` once. The original ten serial actions coexist with standard Open/Save flight actions. Binding signals again during periodic refresh would duplicate transmissions and must be avoided. The daylight toggle updates the shared palette, stylesheet and plot/widget colors. New custom-drawn labels should use the same font and palette conventions.

The global application font is Lucida Grande, registered from the packaged font collection by `configure_fonts`. The collection is loaded privately; the app does not install a system font. Plot axis labels and painter text use the same family. The report itself uses Latin Modern to match the requested LaTeX report style; these are separate presentation choices.

22.2. Where the antenna's physical description is defined

The physical drawing is procedural code inside `src/rocket_gnc_monitor/widgets.py`, specifically `MountView.paintEvent`. There is no separate CAD/mesh file. Local helper functions `box`, `rod` and `face` construct polygons; `palette` defines timber, frame, metal, grid, dark enclosure and other material colors. Faces are depth-sorted and projected with `QPainter`, so the drawing does not require an OpenGL/WebGL viewer.

The local transform functions deliberately separate attachment levels:

- `identity`: fixed stand/legs.
- `turn`: azimuth rotation for turntable and supporting structure.

- **carrier**: azimuth times elevation rotation, then translation to the pivot at renderer position (0, 3.28, 0).
- **avenger**: carrier-local offset (−1.03, 0, 0.16) for the enclosed Avenger assembly.

The common beam is centered at carrier-local zero with extents $2.65 \times 0.10 \times 0.10$ in illustrative model units. The Yagi boom is at local $x = 1.03$, running approximately from $z = -0.35$ to 3.55 , with 21 crossed element positions. The grid is sampled procedurally; the feed extends along local $+Z$ from $z = 0.27$ to 1.04 , in grid material. The dashed boresight continues on the same axis. These values define appearance, not fabrication dimensions.

`mount_rotations` and `MOUNT_COMPASS` implement the corrected basis: renderer $+Y$ up, $+Z$ north, $-X$ east. Do not flip only the compass letters to compensate for a reflection elsewhere. The regression checks $E \times N = U$, headings at the cardinal azimuths and determinant $+1$.

Camera yaw/pitch start at $72^\circ/20^\circ$. Dragging changes those at 0.45 degrees per pixel. Zoom is clamped to 0.65–2.5. The analytic right/up basis stays defined through top/bottom views and full orbit. Camera state and `set_pose` are separate; an orbit event must never call pointer dispatch.

22.3. Changing the model without changing hardware behavior

Edit only geometry/material/attachment transforms for visual changes. Keep attachment objects on **carrier** unless the physical part truly belongs to the fixed stand or azimuth-only support. Check front/side/top/perspective views, full orbit and zoom, feed/boresight collinearity, and the compass tests. Hardware behavior belongs in the controller/protocol layers and requires separate regression evidence.

23. Developer setup and native packaging

23.1. Source environment

Run the following from the application root, not this report directory:

```
make setup PYTHON=python3.12
make run
make demo
make test
make lint
```

Setup creates `.venv`, installs pinned build requirements and installs the package editable without resolving a second dependency set. With `uv`, an existing workflow can use `uv sync -locked`. Runtime simulation additionally needs the staged private Java/engine/bridge.

The shared Makefile uses `.venv/bin/python` on Unix-like hosts and `.venv/Scripts/python.exe` on Windows. The default target prints help. `make clean` removes generated app `build/dist`; it is not a session-data purge command. Preserve validation logs you want to retain before cleaning the application build directory.

23.2. Runtime and bridge

```
make runtime
make bridge OPENROCKET_JAR="/absolute/path/to/swing-24.12.RC.01-all.jar"
```

`runtime` selects OS and process architecture, downloads a Temurin Java 17 JRE, verifies the publisher-provided SHA-256 and stages `vendor/java`. Its manifest records the version, source, hash, OS and architecture. Reuse is allowed only for a matching target. Stage dependencies/runtime in advance for offline builds; do not copy a Mac virtual environment or JRE to Windows.

`bridge` needs JDK 17+ tools `javac` and `jar`, and the team's complete fork JAR. It compiles with Java 17 release and UTF-8 settings, builds `rocket-bridge.jar`, records engine/bridge hashes and copies bridge source into the vendor tree. With a previously staged engine, later `make bridge` can reuse it. Rebuild the bridge after editing its Java source.

23.3. Native platform builds

```
# On macOS, after setup:
make -f Makefile.macos all \
  OPENROCKET_JAR="/absolute/path/to/swing-24.12.RC.01-all.jar"

# On Windows with GNU Make and x64 Python:
make -f Makefile.windows all \
  OPENROCKET_JAR="C:/OpenRocket/swing-24.12.RC.01-all.jar"
```

Each platform's `all` target runs tests, stages runtime, builds bridge, packages and verifies the package. PyInstaller builds on its target OS; this is not cross-compilation. Windows ARM hosts must use x64 Python for the supported emulated build path; runtime architecture follows the Python process rather than host hardware alone.

Without GNU Make, Windows PowerShell can call the scripts directly:

```
py -3.12 scripts/build.py setup
.venv\Scripts\python.exe -m pytest -q
.venv\Scripts\python.exe scripts/build.py runtime
.venv\Scripts\python.exe scripts/build.py bridge --engine "C:\OpenRocket\swing-all.jar"
.venv\Scripts\python.exe scripts/build.py package --platform Windows
.venv\Scripts\python.exe scripts/verify_package.py
```

The prior Windows VM used Python 3.13 x64; use the installed matching launcher version. This historical validation does not assert that the newest Mac feature revision has been rebuilt there.

23.4. Package contents and artifacts

`packaging/RocketGNCMonitor.spec` includes the resources, in-app docs and vendor tree, FFmpeg data, package metadata and the native MGRS library. It excludes unneeded Qt WebEngine/QML/Quick and scientific packages. The launcher is windowed. macOS bundling declares camera usage and the application identity.

The build renders the repository SVG icon into native icons, gathers dependency license/notice files, writes `vendor/build-manifest.json` and retains FFmpeg build information. macOS produces `dist/RocketGNCMonitor.r.app` and an architecture-labeled DMG. Windows produces a complete `dist/RocketGNCMonitor` folder and ZIP. Distributable archives have SHA-256 sidecars.

`MAC_SIGN_IDENTITY` can select a team Developer ID identity at build time; the current local build uses ad-hoc signing. Notarization/stapling and Windows release signing are external release steps. Preserve existing third-party notices, the proprietary font provenance, Java legal files, engine and bridge source/provenance materials. The exact corresponding engine source remains an explicit release record to resolve; this documentation does not grant new licenses or credentials.

24. Testing, verification and evidence

24.1. Test organization

Test group	Contract exercised
<code>test_protocol.py</code> , <code>test_serial.py</code>	Decoder corruption/fragmentation, clock and sequence behavior, pointer bytes and OS pseudo-terminal transport.
<code>test_parity.py</code>	Captured old-UI commands/CSV/shortcuts, actual Qt key events, independent ports and retained confirmations.
<code>test_controller.py</code>	Startup locks, source isolation, stale tracking, dispatch/pending state and mode behavior.
<code>test_domain.py</code> , <code>test_location.py</code>	Geographic/datum/wind validation, MGRS/Plus Codes, preset and coordinate persistence.
<code>test_demo.py</code> , <code>test_legacy.py</code>	Exact bundled logs, timing/seek/quality behavior, safe CSV conversion and provenance.
<code>test_recording.py</code> , <code>test_session_flow.py</code>	SQLite/raw/CSV recovery, pending-batch drain, record-stop-replay and video seek.

Test group	Contract exercised
<code>test_dual_video.py</code>	Camera reservations and dropdown exclusion, independent streams/-failures, legacy replay, dual-video archive round trip and minimum-size panels.
<code>test_flight.py</code>	Portable assets, active snapshots, full-demo and buffer exports, corruption/path rejection, atomic failure and final video inclusion.
<code>test_media.py</code>	FFmpeg arguments, capture/recording lifecycle and available runtime checks.
<code>test_ui.py</code>	Workspace rendering, legacy values/readiness, orbit independence, compass chirality and pseudo-terminal radio status.
<code>test_simulation.py</code>	Bundled N8406 identity, real engine trajectory, named missing-motor and unpowered errors. Engine-dependent cases skip if runtime/bridge is absent.
<code>test_build.py</code>	Windows-on-ARM host with x64 Python selects the x64 runtime correctly.

Run focused tests for the seam being changed, then the appropriate full release checks. Offscreen Mac/Linux Qt tests can use:

```
QT_QPA_PLATFORM=offscreen .venv/bin/python -m pytest -q
.venv/bin/python -m ruff check src tests --select F
git diff --check
```

Windows tests should exercise the native Qt Windows platform; its offscreen font behavior is not equivalent to checking a real native window. These serial tests use pseudo-terminals or in-memory substitutes, not physical boards or recovery hardware.

24.2. Legacy parity baseline

`tests/fixtures/legacy_reference.json` was captured from the original `UI.py`, `rocket.py` and `pointer.py`. It records source hashes and 118 rocket command cases, eight pointer cases, geographic tracking cases, all 43 CSV columns and the ten original shortcut strings. The fixture generator strips physical connection behavior and substitutes in-memory streams. Do not regenerate it merely to make a failing test accept a changed protocol; first establish the intended compatibility decision.

The retained compatibility details include receiver-height division, five-decimal frozen GPS, ground height zero for tracking, servo degree conversion/rounding and two unused padded pyro list entries in good-packet CSV. Documented fixes include rejected-packet counter accuracy, bad-packet-only field inconsistencies, eight BMS columns and duplicate old shortcut connections.

24.3. Packaged verification

```
make verify-package
.venv/bin/python scripts/verify_package.py \
--model /absolute/path/to/zephy_testlaunch.ork \
--output build/package-verification
```

The script launches the actual packaged executable with a minimal system PATH and no developer Python/Java overrides. It checks locked LIVE startup, Lucida Grande, coordinate conversion, planning/recorded flight round trips with URRG, all three demo logs plus video, and the bundled example OpenRocket run. The optional model flag adds a supplied model to the same packaged worker test.

CLI smoke entry points are:

Flag	Use
<code>-demo</code>	Explicit recorded demo startup.
<code>-demo-station GS1 GS2 GS3</code>	Receiver selection; default GS2.
<code>-data-dir PATH</code>	Override per-user application data directory.
<code>-startup-smoke</code>	Check disconnected/locked LIVE startup and exit.
<code>-smoke-test</code>	Exercise recorded demo and synthetic video, write report, exit.

Flag	Use
<code>-flight-smoke</code>	Save/load planning and GS2 flights, URRG/assets/reference, paused isolated replay.
<code>-simulation-smoke MODEL</code>	Exercise private engine with synthetic zero-coordinate site and write report.
<code>-screenshot PATH</code>	Optional native render for smoke verification; not used for this report's diagrams.

24.4. Recorded results and their limits

The September 21 dual-video revision passed 77 Mac regression tests. New checks cover camera reservations during open/close, reciprocal camera and serial dropdown exclusion, independent stream failure and frames, old single-video replay, two-feed flight archives and compact layouts. The refreshed package smoke report records Digital and Analog frame counts separately.

The retained September 20 Mac validation reports 71 passing regression tests, followed by focused final checks. Packaged flight verification restored all 5,464 GS2 samples, URRG, a copied model and reference in paused REPLAY with no physical worker. The bundled example simulation returned 1,551 rows. The supplied Zephyrus ORK returned 11,626 rows in the package's synthetic zero-coordinate smoke setup; this is a dependency/configuration test rather than the actual launch prediction.

The prior Windows 11 ARM VM exercised an x64 package under emulation with 30 tests passing and one POSIX-only skip, before the current revision. Mac signature and DMG checks establish package integrity, not notarization or hardware qualification. The copied evidence in this report folder preserves the original result rather than relabeling it as a fresh test run.

25. Extension recipes and maintenance boundaries

25.1. Add another launch-site preset

Add the name/code to `LaunchLocation`'s preset selector and its selection handler, use the existing converter, emit `preset_selected` and preserve canonical coordinates. Keep elevations explicit. Add a test for exact input provenance, save/load stability, display-format changes, manual edit clearing and cancellation. If presets grow, extracting a data-only catalog is a natural refactor; do not introduce a required network geocoder for offline presets.

25.2. Support the new three-axis telemetry board

Define a versioned wire contract first: framing, checksum/CRC, units, axes, timestamps, reset behavior, sequence numbering and channel identities. Implement a separate decoder/adaptor that produces the established `Sample` contract. Populate measured/demand fields only when the firmware supplies them. Do not reinterpret legacy servo channel numbers as canards/tabs without an explicit hardware mapping.

Add golden packet vectors, fragmented/corrupt/reboot tests, recording and archive round trips, replay/UI tests and captured new-board data. Preserve the old Zephyrus adapter so existing logs and hardware retain their behavior. Update the protocol documentation and qualification status with the code.

25.3. Add a command or pointer feedback protocol

Keep pure encoding in the protocol/command modules and dispatch authorization in the controller. Add the appropriate UI control and confirmation behavior, reference bytes and no-duplicate-shortcut tests. Define acknowledgment IDs, timeouts and partial-write semantics explicitly before showing a new *confirmed* status.

For measured pointer feedback, create a structured RX parser and a separate measured-pose field. Do not overwrite the meaning of the last-sent target. Mechanical homing, cable limits and motion stop require an agreed firmware and operational contract, not merely a renamed host button.

25.4. Extend a saved schema

For an optional mission field, add a dataclass default, validation, editor if needed, and mission/flight/session tests. Older version-1 files rely on default values for absent known fields. Breaking semantic changes need a version bump and explicit migration or rejection; filtering unknown fields is not migration.

For new sample shapes or archive files, update `Sample`, import adapters, record/export paths, `flight.load_flight` validation and replay consumers. Exercise save during active logging, failure before replacement and reopening without original source assets. Preserve queue bounds and authoritative capture independence from UI history.

25.5. Change simulation physics

Modify the Java bridge only against the matching fork API and rebuild `rocket-bridge.jar`. Record solver/options, tests and model/motor hashes. Controlled canard/tab dynamics need explicit model parameters and physics qualification; do not simply clear the `controlled_model_validated:false` flag or infer missing vehicle feedback from demo traces.

25.6. A practical change checklist

Before distributing a change, review the affected source-mode behavior and resource ownership; run meaningful focused tests; retain legacy parity where applicable; inspect affected windows at 1120 by 800 and larger sizes; rebuild the native package; run packaged verification; and update operator/developer docs and release evidence. Physical interface or control changes additionally require the team's hardware acceptance process. Never use a real recovery command as a software smoke test.

A. Complete mission configuration reference

The table follows the current `Mission` dataclass, including retained compatibility fields. Mission JSON adds `schema_version:1`; the default values below are application defaults, not a recommended launch configuration. The validator applies finite-number/range checks where defined; UI spinbox limits and dataclass validation are not identical for every field.

Field	Default	Meaning
<code>name</code>	Untitled mission	Human-readable mission name.
<code>latitude</code>	0.0	WGS84 launch latitude in degrees; -90 to 90.
<code>longitude</code>	0.0	WGS84 launch longitude in degrees; -180 to 180.
<code>altitude</code>	0.0	Launch ellipsoid altitude in metres; used by geographic transforms.
<code>altitude_msl</code>	0.0	Launch mean-sea-level elevation in metres; weather and engine atmosphere.
<code>site_configured</code>	False	Explicit origin-established flag; required for LIVE trajectory and simulation.
<code>launch_location_format</code>	latlon	Entry provenance: latlon, mgrs or pluscode.
<code>launch_location_code</code>	(empty)	Entered/resolved code; up to 80 characters. Canonical lat/lon remains authoritative.
<code>launch_site_name</code>	(empty)	Preset name, currently URRG or empty for Custom; up to 80 characters.
<code>legacy_altitude</code>	unknown	unknown, ellipsoid, <code>gps_agl</code> or <code>barometric_agl</code> ; selects trajectory height interpretation.
<code>canard_count</code>	4	Number of UI canard slots, integer 0 to 16; four tabs are always added.
<code>pointer_latitude</code>	0.0	Retained configuration field; not the current frozen legacy tracking origin.
<code>pointer_longitude</code>	0.0	Retained configuration field; not the current frozen legacy tracking origin.
<code>pointer_altitude</code>	0.0	Retained configuration field; legacy pointing uses ground height zero.
<code>pointer_calibrated</code>	False	Retained assertion, always cleared by disk load and mode changes; not a manual-send prerequisite.
<code>pointer_az_offset</code>	0.0	Retained offset field; not applied by current legacy manual/tracking dispatch.
<code>pointer_el_offset</code>	0.0	Retained offset field; not applied by current legacy manual/tracking dispatch.
<code>pointer_el_min</code>	0.0	Stored route-helper envelope minimum; not an enforced operator control limit.
<code>pointer_el_max</code>	80.0	Stored route-helper envelope maximum; ordered within -90 to 90 degrees.
<code>pointer_az_min</code>	0.0	Stored route-helper azimuth minimum; not enforced by the legacy dispatch path.
<code>pointer_az_max</code>	359.0	Stored route-helper azimuth maximum; ordered within 0 to 360 degrees.
<code>pointer_full_rotation</code>	False	Stored route-helper choice; not a firmware cable-wrap guarantee.
<code>freshness</code>	2.0	Maximum acceptable sample age, 0.1 to 60 seconds.
<code>video_offset</code>	0.0	Seconds; positive value delays recorded video relative to telemetry.
<code>wind</code>	One calm layer at height 0	1 to 200 increasing nonnegative AGL heights with east-/north velocity components; max speed 150 m/s.
<code>wind_source</code>	Manual · calm	Human-readable profile provenance.
<code>weather_raw</code>	{}	Cached normalized request/result/provider payload, when retrieved.
<code>model</code>	(empty)	Selected ORK path. Archives embed it and rebind the extracted path.
<code>motor_files</code>	[]	Up to 100 custom curve paths; selected files are embedded in flight archives.
<code>simulation_index</code>	0	Saved ORK simulation, zero-based integer 0 to 1000.

Field	Default	Meaning
<code>rail_length</code>	2.0	Launch rod length in metres; greater than 0 and at most 100.
<code>rail_tilt</code>	0.0	Tilt from vertical in degrees; at least 0 and less than 90.
<code>rail_heading</code>	0.0	True-north bearing in degrees; UI range 0 to 359.99.
<code>seed</code>	42	Integer random seed, 0 to 2147483647.
<code>low_battery</code>	9.0	Total-battery alert threshold in volts; 1-second dwell and 0.3 V clearing hysteresis.

B. Complete normalized sample reference

`Sample.from_dict` retains known dataclass fields and uses defaults for absent optional values. The file formats add their own structural validation; the dataclass constructor alone is not a complete untrusted-input validator. Vector and dictionary meanings are defined by the producing protocol adapter.

Field	Default	Meaning
<code>t</code>	required	Device uptime seconds after wrap extension, or imported equivalent; align separately to flight zero.
<code>sequence</code>	required	Packet counter; LIVE wire counter is uint16.
<code>source</code>	required	Provenance such as LIVE, DEMO or LEGACY_CSV. REPLAY is a controller mode, not a rewrite of sample source.
<code>received</code>	runtime factory	Host monotonic receive time in seconds; replay refreshes this for current display freshness.
<code>utc</code>	runtime factory	Unix reception timestamp in seconds; original UTC retained for imported/demo data.
<code>altitude</code>	None	Reported filtered barometric altitude, metres, or unavailable.
<code>velocity</code>	None	Reported integrated velocity, metres/second, or unavailable.
<code>latitude</code>	None	Rocket WGS84 latitude in degrees, or unavailable.
<code>longitude</code>	None	Rocket WGS84 longitude in degrees, or unavailable.
<code>gps_altitude</code>	None	Reported GPS height in metres, with explicit mission interpretation.
<code>gps_fix</code>	0	GPS fix code; geographic display requires at least 3.
<code>attitude</code>	None	Optional three-element roll/pitch/yaw integrated rotations, degrees; not a qualified attitude solution.
<code>rates</code>	None	Optional XYZ angular rates, degrees/second; legacy Y sign is retained.
<code>acceleration</code>	None	Optional XYZ acceleration, metres/second squared.
<code>enu</code>	None	Optional local east/north/up metres; frame/provenance must be known.
<code>battery</code>	None	Optional total battery volts, sum of the three cells for LIVE.
<code>rss_i</code>	None	Optional receiver RSSI in dBm.
<code>phase</code>	Unknown	Received or imported state label; unknown codes remain visible.
<code>actuators</code>	runtime factory	Dictionary by channel name; demand/measured angles or legacy drive when actually available.
<code>details</code>	runtime factory	Extensible decoded legacy values, boot/time metadata, receiver data, raw CSV provenance and quality notes.

C. Legacy CSV columns and derived values

This is the exact 43-column order written by the compatibility adapter. The rich `SessionReader.export_csv` format is different and includes full sample JSON. Imported records retain their original row text and available decoded values. List padding described below refers to live good-packet output; it should not be used to invent extra hardware channels.

Column	Field	Meaning
1	timestamp	Receive UTC, Unix seconds.
2	pyros	Six pyro continuity codes.
3	servos	Four legacy drive values.
4	servos_deg	Four converted servo angles; first two use 60-degree scale, last two 50-degree scale; good CSV rounded to two decimals.
5	accelerometer	XYZ acceleration, m/s squared.
6	barofilteredalt	Filtered barometric altitude, m.
7	temp	Legacy calibrated temperature, degrees C.
8	gyro	XYZ gyro rates, degrees/s.
9	gps_fix	Rocket fix code.
10	lat	Rocket latitude, degrees.
11	lon	Rocket longitude, degrees.
12	gpsalt	GPS height, m; datum depends on firmware/configuration.
13	gps_horiz_prec	Horizontal GPS accuracy, m.
14	gps_vert_prec	Vertical GPS accuracy, m.
15	gps_num_sat	Satellite count.
16	flight_time	Original raw device clock, milliseconds.
17	yaw_gyro_int	Integrated yaw, degrees.
18	pitch_gyro_int	Integrated pitch, degrees.
19	roll_gyro_int	Integrated roll, degrees.
20	state	Legacy state enum string.
21	pktnum	Packet counter.
22	rsssi	Ground receiver RSSI, dBm.
23	armed_pyros	Armed bits as list; good-packet CSV retains two unused zero entries.
24	fired_pyros	Fired bits as list; same good-packet padding.
25	badpackets	Rejected-packet counter.
26	rxrsssi	Onboard RSSI, dBm.
27	accel_integrated_velo	Reported integrated velocity, m/s.
28	baro_max_alt	Maximum barometric altitude, m.
29	gps_max_alt	Maximum GPS altitude, m.
30	pyro_resistances	Pyro resistance list; good-packet output pads two unused zero entries.
31	cell_voltages	Three cell voltages, V.
32	total_current	Total current, A.
33	converter_voltages	Six rail voltages, V.
34	converter_currents	Six rail currents, A.
35	bms_protections_enabled	BMS enabled protection bits.
36	bms_protection_status	BMS triggered protection bits.
37	bms_temp	BMS temperature, degrees C.
38	enabled_status	Derived rail-enabled states from voltage proximity to nominal.
39	angleFromVertical	Legacy angle-from-vertical derived from pitch/yaw, degrees.
40	gnd_lat	Frozen/display receiver latitude convention, rounded to five decimals.
41	gnd_lon	Receiver longitude convention, rounded to five decimals.
42	gnd_fix	Legacy ground fix Boolean: trailer fix equals 3.
43	gnd_alt	Legacy receiver height: signed wire integer divided by 1000, rounded for display/CSV.

The live servo degree conversion is

$$\alpha_i = \frac{d_i - 1500}{500} \begin{cases} 60^\circ, & i = 0, 1, \\ 50^\circ, & i = 2, 3. \end{cases}$$

The legacy angle from vertical is $\arccos(\cos p \cos y)$, with the cosine product clamped to $[-1, 1]$. The raw temperature calibration in `legacy_values` is

$$T = \frac{2000 + (T_{\text{raw}} - 0x91E3 \cdot 256)0x6FEC/2^{23}}{100}.$$

These formulas reproduce the former monitor; they do not establish new sensor calibration accuracy.

D. Worked file examples

The `examples` directory accompanies the source report. Its inputs have been loaded through the application's validators as documentation examples.

D.1. URRG planning template

`examples/urrg-mission.json` includes all current mission fields and the exact URRG code/canonical coordinates. It deliberately leaves `site_configured:false`, altitude convention unknown and both elevations zero so the example cannot be mistaken for a surveyed setup. Open it with **Open mission**, then review and complete those values. The essential location portion is:

```
{
  "schema_version": 1,
  "name": "URRG template - verify launch elevations",
  "launch_site_name": "URRG",
  "launch_location_format": "mgrs",
  "launch_location_code": "18TUN2061530290",
  "site_configured": false,
  "legacy_altitude": "unknown"
}
```

The full file also supplies the decoded latitude and longitude; do not treat this shortened excerpt as a replacement for the full template.

D.2. Wind profile

```
{
  "schema_version": 1,
  "source": "Documentation example only",
  "layers": [
    {
      "height": 0,
      "east": 0,
      "north": 0
    },
    {
      "height": 1000,
      "east": 3,
      "north": -1
    }
  ]
}
```

The second layer has eastward and southward components. This is a formatting example, not a retrieved URRG forecast.

D.3. Normalized trajectory

```
time_s,east_m,north_m,up_m
0,0,0,0
1,1,2,10
2,2,4,20
```

The matching manifest is:

```
{
  "schema_version": 1,
  "frame": "ENU",
  "units": "m,s",
  "name": "Documentation geometry example",
  "altitude_datum": "launch_relative",
  "origin": [
    42.70416776971639,
    -77.19029501648315,
    0
  ],
  "synthetic": true,
  "controlled_model_validated": false
}
```

The example is explicitly synthetic and is therefore rejected as a LIVE reference. It illustrates file structure and can be used for offline tests.

D.4. Portable flight metadata excerpt

```
{
  "format": "rocket-gnc-flight",
  "schema_version": 1,
  "app_version": "0.1.0",
  "source_mode": "DEMO",
  "scope": "Complete Zephyrus GS2 dataset",
  "session": "session",
  "samples": 5464,
  "position": 174.0,
  "flight_zero": 747.766,
  "time_aligned": true,
  "files": {
    "mission.json": {
      "size": "<actual byte count>",
      "sha256": "<actual content hash>"
    }
  }
}
```

This explanatory excerpt omits other entries and contains descriptive placeholders; it is not an importable archive. Use the app’s Save flight operation to create a valid inventory and package.

E. Source and validation index

The code is the behavioral authority for this report. The following retained documents supplement the source map and explain provenance and compatibility:

Source	Role
docs/OPERATOR_GUIDE.md	In-app operator help; this report expands it into full functionality and developer coverage.
docs/LEGACY_PARITY.md	Original UI inventory, captured vectors, exact shortcuts and compatibility exceptions.
docs/PROTOCOL.md	Audited telemetry, pointer and uplink wire contract.
docs/DEMO_DATA.md	GS1/GS2/GS3 source identity, timing and known quality issues.
docs/BUILD.md	Native build, runtime staging, offline preparation and signing workflow.
docs/IMPLEMENTATION_STATUS.md	Current versus historical acceptance status.

Source	Role
<code>docs/THIRD_PARTY_NOTICES.md</code>	Bundled dependency notices and provenance references.
<code>resources/fonts/README.md</code>	User-designated font resource and unchanged source hash.
<code>resources/demo/manifest.json</code>	Original CSV hashes, row counts, durations and launch alignment.
<code>resources/motors/manifest.json</code>	Bundled motor resource integrity and source identity.
<code>tests/fixtures/legacy_reference.json</code>	Captured original protocol/CSV/shortcut behavior.
<code>evidence/source-manifest.json</code>	Exact reviewed source/resource bytes and report-style reference hash.
<code>evidence/application-validation-2026-09-20.json</code>	Copied existing full/focused and packaged validation summary; preserves original test context.

The original design plans describe intentions and earlier stages; they are not used here to claim unimplemented capabilities. In particular, active legacy manual pointing does not enforce the early proposed software envelope, current DEMO uses recorded data rather than the synthetic fixture, and current Windows acceptance is older than this Mac revision.

F. Glossary

Term	Meaning in this app
AGL	Height above the configured launch/ground reference.
Ellipsoid height	Height relative to the WGS84 ellipsoid; used in ECEF/ENU transforms.
MSL	Mean sea level elevation; entered separately for atmosphere and weather.
ECEF / ENU	Earth-centered Earth-fixed / local east-north-up coordinate systems.
GNC	Guidance, navigation and control; this app monitors available data and sends legacy commands.
MGRS / Plus Code	Offline geographic coordinate representations; neither supplies elevation.
Mission	Configuration object; mission JSON alone can contain external asset paths.
Session	Directory containing indexed recorded data, raw bytes, CSVs, events and optional video/reference.
Flight file	Portable versioned ZIP64 archive with mission/assets and available recording/reference data.
Snapshot	Consistent committed subset saved while acquisition continues; active video segment may be absent.
Nominal reference	OpenRocket output under the recorded options; not qualified controlled-vehicle dynamics.
Sent / acknowledged	Serial dispatch / confirmed device response. The legacy command paths provide the first, not the second.
Source mode / phase	LIVE, DEMO or REPLAY / received onboard rocket state; independent concepts.
WAL	SQLite write-ahead log; retain it with a crashed source session during recovery.